

Spring 核心

Bean 的实例化方法

在 Spring 的 IoC 容器中，Bean 的实例化是容器的核心工作之一，IoC 容器负责根据配置规则创建 Bean 对象并管理其生命周期。在入门程序中已经验证：Spring 默认采用无参数构造方法创建 Bean，这也是开发中最常用的方式；但实际开发中，有些类没有无参构造、或需要通过工厂统一创建对象，因此 Spring 提供了3种核心的 Bean 实例化方法，再加上 Spring 特有的FactoryBean 接口实例化，共 4 种常用方式，本次逐一讲解。

Spring为Bean提供了多种实例化方式，通常包括4种方式。（也就是说在Spring中为Bean对象的创建准备了多种方案，目的是：更加灵活）

- 第一种：通过构造方法实例化
- 第二种：通过简单工厂模式实例化
- 第三种：通过factory-bean实例化
- 第四种：通过FactoryBean接口实例化

通过构造方法实例化

我们之前一直使用的就是这种方式。默认情况下，会调用Bean的无参数构造方法。

- User.java

```
package org.cqipu.edu.bean;

public class User {
    public User() {
        System.out.println("User类的无参数构造方法执行。");
    }
}
```

- spring.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="userBean" class="org.cqipu.edu.bean.User"/>
</beans>
```

- 测试程序

```
package org.cqipu.edu;
import org.cqipu.edu.bean.User;
import org.junit.Test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class SpringInstantiationTest {

    @Test
    public void testConstructor(){
        ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring.xml");
    }
}
```

```

        User user = applicationContext.getBean("userBean", User.class);
        System.out.println(user);
    }
}

```

测试结果

✓ 测试 已通过: 1共 1 个测试 - 533毫秒

D:\Java21\bin\java.exe ...

User类的无参数构造方法执行。

org.cqipu.edu.bean.User@6995bf68

通过简单工厂模式实例化

- 第一步：定义一个Bean
 - Vip.java

```

package org.cqipu.edu.bean;

public class Vip {
}

```

- 第二步：编写简单工厂模式当中的工厂类
 - VipFactory

```

package org.cqipu.edu.bean;

public class VipFactory {
    public static Vip get(){
        return new Vip();
    }
}

```

- 第三步：在Spring配置文件中指定创建该Bean的方法（使用factory-method属性指定）
 - spring.xml

```

<bean id="vipBean" class="org.cqipu.edu.bean.VipFactory" factory-method="get"/>

```

- 第四步：编写测试程序
 - 测试程序

```

@Test
public void testSimpleFactory(){
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring.xml");
    Vip vip = applicationContext.getBean("vipBean", Vip.class);
    System.out.println(vip);
}

```

✓ 测试 已通过: 1共 1 个测试 - 411毫秒

D:\Java21\bin\java.exe ...

User类的无参数构造方法执行。

org.cqipu.edu.bean.Vip@53dbe163

通过factory-bean实例化

这种方式本质上是：通过工厂方法模式进行实例化。

- 第一步：定义一个Bean
 - Order.java

```
package org.cqipu.edu.bean;

public class Order {
}
```

- 第二步：定义具体工厂类，工厂类中定义实例方法
 - OrderFactory

```
package org.cqipu.edu.bean;

public class OrderFactory {
    public Order get(){
        return new Order();
    }
}
```

- 第三步：在Spring配置文件中指定factory-bean以及factory-method
 - spring.xml

```
<bean id="orderFactory" class="org.cqipu.edu.bean.OrderFactory"/>
<bean id="orderBean" factory-bean="orderFactory" factory-method="get"/>
```

- 第四步：编写测试程序

```
@Test
public void testSelfFactoryBean(){
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring.xml");
    Order orderBean = applicationContext.getBean("orderBean", Order.class);
    System.out.println(orderBean);
}
```

✓ 测试 已通过: 1共 1 个测试 - 429毫秒

D:\Java21\bin\java.exe ...

User类的无参数构造方法执行。

org.cqipu.edu.bean.Order@4b14918a

通过FactoryBean接口实例化

以上的第三种方式中，factory-bean是我们自定义的，factory-method也是我们自己定义的。

在Spring中，当你编写的类直接实现FactoryBean接口之后，factory-bean不需要指定了，factory-method也不需要指定了。

factory-bean会自动指向实现FactoryBean接口的类，factory-method会自动指向getObject()方法。

- 第一步：定义一个Bean
 - Person.java



```
package org.cqipu.edu.bean;

public class Person {
}
```

- 第二步：编写一个类实现FactoryBean接口
 - PersonFactoryBean.java



```
package org.cqipu.edu.bean;

import org.springframework.beans.factory.FactoryBean;

public class PersonFactoryBean implements FactoryBean<Person> {

    @Override
    public Person getObject() throws Exception {
        return new Person();
    }

    @Override
    public Class<?> getObjectType() {
        return null;
    }

    @Override
    public boolean isSingleton() {
        // true表示单例
        // false表示原型
        return true;
    }
}
```

- 第三步：在Spring配置文件中配置FactoryBean
 - spring.xml



```
<bean id="personBean" class="org.cqipu.edu.bean.PersonFactoryBean"/>
```

测试程序

```
@Test
```

```
public void testFactoryBean(){
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring.xml");
    Person personBean = applicationContext.getBean("personBean", Person.class);
    System.out.println(personBean);

    Person personBean2 = applicationContext.getBean("personBean", Person.class);
    System.out.println(personBean2);
}
```

```
D:\Java21\bin\java.exe ...
```

```
User类的无参数构造方法执行。
```

```
org.cqipu.edu.bean.Person@176b3f44
```

```
org.cqipu.edu.bean.Person@176b3f44
```

FactoryBean在Spring中是一个接口。被称为“工厂Bean”。“工厂Bean”是一种特殊的Bean。所有的“工厂Bean”都是用来协助Spring框架来创建其他Bean对象的。

BeanFactory和FactoryBean的区别

BeanFactory

Spring IoC容器的顶级对象，BeanFactory被翻译为“Bean工厂”，在Spring的IoC容器中，“Bean工厂”负责创建Bean对象。

BeanFactory是工厂。

FactoryBean

FactoryBean：它是一个Bean，是一个能够辅助Spring实例化其它Bean对象的一个Bean。

在Spring中，Bean可以分为两类：

- 第一类：普通Bean
- 第二类：工厂Bean（记住：工厂Bean也是一种Bean，只不过这种Bean比较特殊，它可以辅助Spring实例化其它Bean对象。）

注入自定义Date

我们前面说过，java.util.Date在Spring中被当做简单类型，简单类型在注入的时候可以直接使用value属性或value标签来完成。但我们之前已经测试过了，对于Date类型来说，采用value属性或value标签赋值的时候，对日期字符串的格式要求非常严格，必须是这种格式的：Mon Oct 10 14:30:26 CST 2022。其他格式是会被识别的。如以下代码：

- Student.java

```
package org.cqipu.edu.bean;
```

```
import java.util.Date;
public class Student {
    private Date birth;
```

```
    public void setBirth(Date birth) {
        this.birth = birth;
    }
```

```
@Override
```

```
public String toString() {
    return "Student{" +
        "birth=" + birth +
```

```
        '}'  
    }  
}
```

- spring.xml

● ● ●

```
<bean id="studentBean" class="org.cqipu.edu.bean.Student">  
    <property name="birth" value="Mon Oct 10 14:30:26 CST 2002"/>  
</bean>
```

- 测试程序

● ● ●

```
@Test  
public void testDate(){  
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring.xml");  
    Student studentBean = applicationContext.getBean("studentBean", Student.class);  
    System.out.println(studentBean);  
}
```

✓ 测试 已通过: 1共 1 个测试 - 483毫秒

D:\Java21\bin\java.exe ...

User类的无参数构造方法执行。

Student{birth=Fri Oct 11 04:30:26 CST 2002}

如果把日期格式修改一下：

- spring.xml

● ● ●

```
<bean id="studentBean" class="org.cqipu.edu.bean.Student">  
    <property name="birth" value="2002-10-10"/>  
</bean>
```

测试结果

● ● ●

```
@Test  
public void testDate(){  
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring.xml");  
    Student studentBean = applicationContext.getBean("studentBean", Student.class);  
    System.out.println(studentBean);  
}
```

D:\Java21\bin\java.exe ...

User类的无参数构造方法执行。

org.springframework.beans.factory.BeanCreationException: Error creating bean with name 'studentBean' defined in class path resource [spring.xml]: Failed to convert property value of type 'java.lang.String' to required type 'java.util.Date' for property 'birth'; Cannot convert value of type 'java.lang.String' to required type 'java.util.Date' for property 'birth': no matching editors or conversion strategy found

这种情况下，我们就可以使用FactoryBean来完成这个骚操作。

编写DateFactoryBean实现FactoryBean接口：

- DateFactoryBean

```

package org.cqipu.edu.bean;
import org.springframework.beans.factory.FactoryBean;
import java.text.SimpleDateFormat;
import java.util.Date;

public class DateFactoryBean implements FactoryBean<Date> {

    // 定义属性接收日期字符串
    private String date;

    // 通过构造方法给日期字符串属性赋值
    public DateFactoryBean(String date) {
        this.date = date;
    }

    @Override
    public Date getObject() throws Exception {
        SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd");
        return sdf.parse(this.date);
    }

    @Override
    public Class<?> getObjectType() {
        return null;
    }
}

```

编写spring配置文件：

- spring.xml

```

<bean id="dateBean" class="org.cqipu.edu.bean.DateFactoryBean">
    <constructor-arg name="date" value="1999-10-11"/>
</bean>

<bean id="studentBean" class="org.cqipu.edu.bean.Student">
    <property name="birth" ref="dateBean"/>
</bean>

```

测试结果

```

@Test
public void testDate(){
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring.xml");
    Student studentBean = applicationContext.getBean("studentBean", Student.class);
    System.out.println(studentBean);
}

```

✓ 测试 已通过: 1共 1 个测试 - 482毫秒

D:\Java21\bin\java.exe ...

User类的无参数构造方法执行。

Student{birth=Mon Oct 11 00:00:00 CST 1999}

Bean 的生命周期

什么是 Bean 的生命周期

生命周期，就跟人一样，有出生、有工作、有死亡。

Spring 其实就是一个管理 Bean 对象的工厂。它负责对象的创建，对象的销毁等。

在 Spring 框架里，一个 Bean 从被 Spring 容器创建出来，到给它的属性赋好值，做一些初始化的准备工作，然后交给我们程序去调用，最后 Spring

容器关闭时把它清理掉，这个完整的全过程，就叫 Bean 的生命周期。

- 接下来我们需要了解
 - 什么时候创建 Bean 对象？
 - 创建 Bean 对象的前后会调用什么方法？
 - Bean 对象什么时候销毁？
 - Bean 对象的销毁前后调用什么方法？

为什么要知道 Bean 的生命周期

其实生命周期的本质是：在哪个时间节点上调用了哪个类的哪个方法。

我们需要充分的了解在这个生命线上，都有哪些特殊的时间节点。

只有我们知道了特殊的时间节点都在哪，到时我们才可以确定代码写到哪。

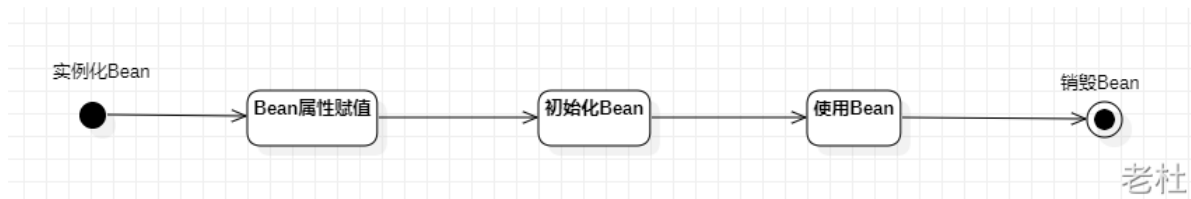
我们可能需要在某个特殊的时间点上执行一段特定的代码，这段代码就可以放到这个节点上。当生命线走到这里的时候，自然会被调用。

Bean 的生命周期之 5 步

Bean 生命周期的管理，可以参考 Spring 的源码：AbstractAutowireCapableBeanFactory 类的 doCreateBean () 方法。

Bean 生命周期可以粗略的划分为五大步：

- 第一步：实例化 Bean
- 第二步：Bean 属性赋值
- 第三步：初始化 Bean
- 第四步：使用 Bean
- 第五步：销毁 Bean



1. 第一步：实例化 Bean

- 类比作“新生命降生”，核心作用是为 Bean 对象在内存中开辟专属的存储空间。
- Spring 容器启动后，会解析 spring.xml 配置文件；当识别到 `<bean>` 标签中配置的全限定类名，底层会通过 Java 反射机制，调用该类的无参构造方法完成对象的创建。
- 注意：如果实体类手动编写了有参构造方法，却忘记补充无参构造方法，Spring 在实例化这一步会直接抛出异常！因为 Spring 默认只通过无参构造方法创建 Bean 实例。

2. 第二步：Bean 属性赋值

- 类比成“为新生儿赋予姓名、穿戴衣物”，对象创建完成后，为其填充专属的属性特征。
- Spring 解析到 `<bean>` 标签内的 `<property>` 配置时，会再次通过 Java 反射机制，调用对象对应的 setter 方法，将配置的属性值传入并赋值。
- 这一步就是 Spring 核心的DI（依赖注入）操作！无论是注入普通字符串等基本数据，还是注入其他 Bean 对象，所有依赖注入的逻辑都在此步骤完成。

3. 第三步：初始化 Bean

- 类比成“正式上岗前的岗前培训”，对象已创建且属性填充完毕，在投入使用前完成最后的准备工作。
- 此时 Bean 对象已实例化且所有属性赋值完成，Spring 会检查配置文件（或对应注解）中是否配置了初始化方法，若存在则会调用该自定义方法，完成 Bean 使用前的准备逻辑。
- 这一步是为 Bean 投入使用做前置准备。例如在第二步通过 DI 注入了数据库连接的 URL、用户名、密码等配置参数，那么在初始化方法中，就可以利用这些参数执行数据库连接的初始化代码——属性赋值只是传递了配置信息，而初始化阶段才是真正执行准备动作、让 Bean 具备实际工作能力的关键环节。

4. 第四步：使用 Bean

- 类比成“正式上岗履职，开始执行业务工作”，Bean 完成全部前置准备后，正式投入业务场景发挥作用。
- 经历实例化、属性赋值、初始化三步后，Bean 已成为状态完备、可直接使用的实例对象。Spring 会将其存入 IoC 容器的底层存储结构，本质是一个名为“单例池”的 Map 集合。当开发者在代码中调用 `context.getBean("user")` 等方法时，Spring 会从容器中取出该 Bean 实例，供业务代码调用其方法完成具体功能。

5. 第五步：销毁 Bean

- 类比成“正式退休 / 生命周期终结”，Bean 完成所有工作后，结束生命周期。
- 当程序运行完毕、Spring 容器正常关闭时，手动调用 `context.close()` 方法，Spring 不会直接销毁 Bean 实例，而是先执行我们预先配置的销毁方法，完成收尾处理后再释放对象。
- 这一步至关重要，核心作用是释放系统资源。例如断开数据库连接、关闭 IO 文件流、销毁线程池等操作都在此执行。若忽略资源清理工作，极易引发内存泄漏问题，影响程序的稳定运行。

编写测试程序：

- 定义一个Bean
 - User

```
package org.cqipu.edu.bean;

public class User {
    private String name;

    public User() {
        System.out.println("1. 实例化Bean");
    }

    public void setName(String name) {
        this.name = name;
        System.out.println("2. Bean属性赋值");
    }

    public void initBean(){
        System.out.println("3. 初始化Bean");
    }

    public void destroyBean(){
        System.out.println("5. 销毁Bean");
    }
}
```

- spring.xml



```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <!--
        init-method属性指定初始化方法。
        destroy-method属性指定销毁方法。
    -->
    <bean id="userBean" class="org.cqipu.edu.bean.User" init-method="initBean" destroy-method="destroyBean">
        <property name="name" value="zhangsan"/>
    </bean>
</beans>
```

- 测试程序



```
package org.cqipu.edu;

import org.cqipu.edu.bean.User;
import org.junit.Test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class BeanLifecycleTest {
    @Test
    public void testLifecycle(){
        ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring.xml");
        User userBean = applicationContext.getBean("userBean", User.class);
        System.out.println("4. 使用Bean");
        // 只有正常关闭spring容器才会执行销毁方法
        ClassPathXmlApplicationContext context = (ClassPathXmlApplicationContext) applicationContext;
        context.close();
    }
}
```

测试结果

```
D:\Java21\bin\java.exe ...
```

1. 实例化Bean
2. Bean属性赋值
3. 初始化Bean
4. 使用Bean
5. 销毁Bean

Bean生命周期之7步

在以上的5步中，第3步是初始化Bean，如果你还想在初始化前和初始化后添加代码，可以加入“Bean后处理器”。

编写一个类实现BeanPostProcessor类，并且重写before和after方法：

- LogBeanPostProcessor



```
package org.cqipu.edu.bean;

import org.springframework.beans.BeansException;
import org.springframework.beans.factory.config.BeanPostProcessor;
```

```
public class LogBeanPostProcessor implements BeanPostProcessor {
    @Override
    public Object postProcessBeforeInitialization(Object bean, String beanName) throws BeansException {
        System.out.println("Bean后处理器的before方法执行，即将开始初始化");
        return bean;
    }

    @Override
    public Object postProcessAfterInitialization(Object bean, String beanName) throws BeansException {
        System.out.println("Bean后处理器的after方法执行，已完成初始化");
        return bean;
    }
}
```

在spring.xml文件中配置“Bean后处理器”：

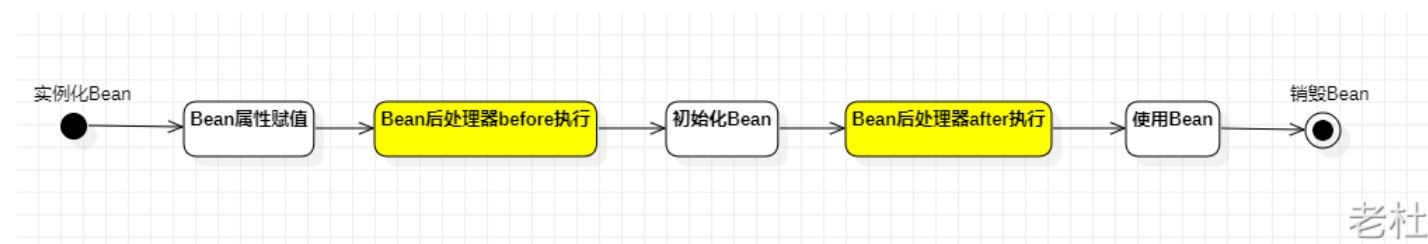
- spring.xml

```
<!-- 配置Bean后处理器。这个后处理器将作用于当前配置文件中所有的bean。 -->
<bean class="org.cqipu.edu.bean.LogBeanPostProcessor"/>
```

一定要注意：在spring.xml文件中配置的Bean后处理器将作用于当前配置文件中所有的Bean。
执行测试程序：

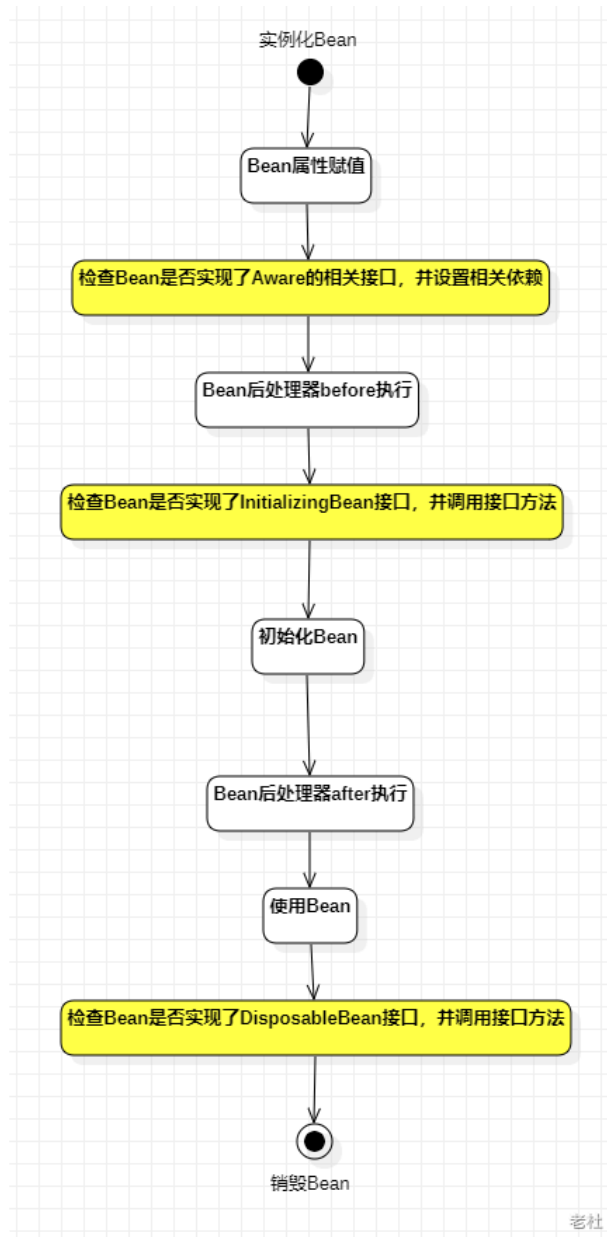
```
D:\Java21\bin\java.exe ...
1. 实例化Bean
2. Bean属性赋值
Bean后处理器的before方法执行，即将开始初始化
3. 初始化Bean
Bean后处理器的after方法执行，已完成初始化
4. 使用Bean
5. 销毁Bean
```

如果加上Bean后处理器的话，Bean的生命周期就是7步了：



Bean生命周期之10步

如果根据源码跟踪，可以划分更细粒度的步骤，10步：



上图中检查Bean是否实现了Aware的相关接口是什么意思？

Aware相关的接口包括：BeanNameAware、BeanClassLoaderAware、BeanFactoryAware

- 当Bean实现了BeanNameAware，Spring会将Bean的名字传递给Bean。
- 当Bean实现了BeanClassLoaderAware，Spring会将加载该Bean的类加载器传递给Bean。
- 当Bean实现了BeanFactoryAware，Spring会将Bean工厂对象传递给Bean。

测试以上10步，可以让User类实现5个接口，并实现所有方法：

- BeanNameAware
- BeanClassLoaderAware
- BeanFactoryAware
- InitializingBean
- DisposableBean

代码如下：

- User.java



```

package org.cqipu.edu.bean;
import org.springframework.beans.BeansException;
import org.springframework.beans.factory.*;

public class User implements BeanNameAware, BeanClassLoaderAware, BeanFactoryAware, InitializingBean, DisposableBean {
    private String name;

    public User() {
        System.out.println("1. 实例化Bean");
    }

    public void setName(String name) {
        this.name = name;
        System.out.println("2.Bean属性赋值");
    }

    public void initBean(){
        System.out.println("6. 初始化Bean");
    }

    public void destroyBean(){
        System.out.println("10. 销毁Bean");
    }

    @Override
    public void setBeanClassLoader(ClassLoader classLoader) {
        System.out.println("3. 类加载器: " + classLoader);
    }

    @Override
    public void setBeanFactory(BeanFactory beanFactory) throws BeansException {
        System.out.println("3.Bean工厂: " + beanFactory);
    }

    @Override
    public void setBeanName(String name) {
        System.out.println("3.bean名字: " + name);
    }

    @Override
    public void destroy() throws Exception {
        System.out.println("9.DisposableBean destroy");
    }

    @Override
    public void afterPropertiesSet() throws Exception {
        System.out.println("5.afterPropertiesSet执行");
    }
}

```

- LogBeanPostProcessor.java

```

package org.cqipu.edu.bean;
import org.springframework.beans.BeansException;
import org.springframework.beans.factory.config.BeanPostProcessor;

public class LogBeanPostProcessor implements BeanPostProcessor {
    @Override
    public Object postProcessBeforeInitialization(Object bean, String beanName) throws BeansException {
        System.out.println("4.Bean后处理器的before方法执行，即将开始初始化");
        return bean;
    }
}

```

```

@Override
public Object postProcessAfterInitialization(Object bean, String beanName) throws BeansException {
    System.out.println("7.Bean后处理器的after方法执行，已完成初始化");
    return bean;
}
}

```

执行结果

✓ 测试 已通过: 1共 1 个测试 - 543毫秒

D:\Java21\bin\java.exe ...

- 1.实例化Bean
- 2.Bean属性赋值
- 3.bean名字: userBean
- 3.类加载器: jdk.internal.loader.ClassLoaders\$AppClassLoader@36baf30c
- 3.Bean工厂: org.springframework.beans.factory.support.DefaultListableBeanFactory@327bcebd: defining beans [userBean,org.cqipu.edu.bean.LogBeanPostProcessor#0]; root of factory hierarchy
- 4.Bean后处理器的before方法执行，即将开始初始化
- 5.afterPropertiesSet执行
- 6.初始化Bean
- 7.Bean后处理器的after方法执行，已完成初始化
- 8.使用Bean
- 9.DisposableBean destroy
- 10.销毁Bean

通过测试可以看出来：

- InitializingBean的方法早于init-method的执行。
- DisposableBean的方法早于destroy-method的执行。

对于SpringBean的生命周期，掌握之前的7步即可。够用。

Bean的作用域不同，管理方式不同

Spring 根据Bean的作用域来选择管理方式。

- 对于singleton作用域的Bean，Spring 能够精确地知道该Bean何时被创建，何时初始化完成，以及何时被销毁；
- 而对于 prototype 作用域的 Bean，Spring 只负责创建，当容器创建了 Bean 的实例后，Bean 的实例就交给客户端代码管理，Spring 容器将不再跟踪其生命周期。

我们把之前User类的spring.xml文件中的配置scope设置为prototype：

- spring02.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <!--
    init-method属性指定初始化方法。
    destroy-method属性指定销毁方法。
    -->
    <bean id="userBean" class="org.cqipu.edu.bean.User" init-method="initBean" destroy-method="destroyBean"
scope="prototype">
        <property name="name" value="zhangsan"/>
    </bean>

    <!-- 配置Bean后处理器。这个后处理器将作用于当前配置文件中所有的bean。-->
    <bean class="org.cqipu.edu.bean.LogBeanPostProcessor"/>
</beans>

```

测试程序

```
● ● ●
@Test
public void testLifecycle02(){
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring02.xml");
    User userBean = applicationContext.getBean("userBean", User.class);
    System.out.println("8. 使用Bean");
    // 只有正常关闭spring容器才会执行销毁方法
    ClassPathXmlApplicationContext context = (ClassPathXmlApplicationContext) applicationContext;
    context.close();
}
```

测试结果

D:\Java21\bin\java.exe ...

- 1.实例化Bean
- 2.Bean属性赋值
- 3.bean名字: userBean
- 3.类加载器: jdk.internal.loader.ClassLoaders\$AppClassLoader@36baf30c
- 3.Bean工厂: org.springframework.beans.factory.support.DefaultListableBeanFactory@327bcebd: defining beans [userBean,org.cqipu.edu.bean.LogBeanPostProcessor#0]; root of factory hierarchy
- 4.Bean后处理器的before方法执行, 即将开始初始化
- 5.afterPropertiesSet执行
- 6.初始化Bean
- 7.Bean后处理器的after方法执行, 已完成初始化
- 8.使用Bean

通过测试一目了然。只执行了前8步, 第9和10都没有执行。

自己new的对象如何让Spring管理

有些时候可能会遇到这样的需求, 某个java对象是我们自己new的, 然后我们希望这个对象被Spring容器管理, 怎么实现?

- User01.java

```
● ● ●
package org.cqipu.edu.bean;

public class User01 {
}
```

测试代码

```
● ● ●
package org.cqipu.edu;

import org.junit.Test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import org.cqipu.edu.bean.User01;
import org.springframework.beans.factory.support.DefaultListableBeanFactory;

public class RegisterBeanTest {

    @Test
    public void testBeanRegister(){
        // 自己new的对象
        User user = new User();
        System.out.println(user);

        // 创建 默认可列表BeanFactory 对象
```

```

DefaultListableBeanFactory factory = new DefaultListableBeanFactory();
// 注册Bean
factory.registerSingleton("userBean", user);
// 从spring容器中获取bean
User userBean = factory.getBean("userBean", User.class);
System.out.println(userBean);
    }
}

```

测试结果

✓ 测试 已通过: 1共 1 个测试 - 385毫秒

D:\Java21\bin\java.exe ...

1. 实例化Bean

org.cqipu.edu.bean.User@2f7a2457

org.cqipu.edu.bean.User@2f7a2457

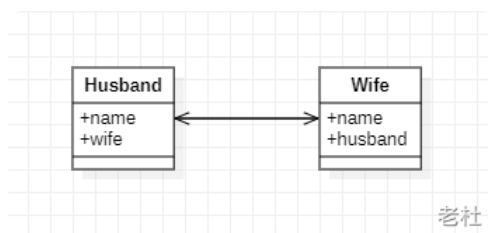
Bean 的循环依赖问题

什么是Bean 的循环依赖

循环依赖（Circular Dependency）指的是两个或多个 Bean 之间互相持有对方的引用，最终形成了一个闭环。

A对象中有B属性。B对象中有A属性。这就是循环依赖。我依赖你，你也依赖我。

比如：丈夫类Husband，妻子类Wife。Husband中有Wife的引用。Wife中有Husband的引用。



- Husband.java

● ● ●

```

package org.cqipu.edu.bean;

public class Husband {
    private String name;
    private Wife wife;
}

```

- Wife.java

● ● ●

```

package org.cqipu.edu.bean;
public class Wife {
    private String name;
    private Husband husband;
}

```

singleton下的set注入产生的循环依赖

我们来编写程序，测试一下在singleton+setter的模式下产生的循环依赖，Spring是否能够解决？

- Husband.java



```
package org.cqipu.edu.bean;

public class Husband {
    private String name;
    private Wife wife;

    public void setName(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    public void setWife(Wife wife) {
        this.wife = wife;
    }

    // toString()方法重写时需要注意: 不能直接输出wife, 输出wife.getName()。要不然会出现递归导致的栈内存溢出错误。
    @Override
    public String toString() {
        return "Husband{" +
            "name='" + name + '\'' +
            ", wife=" + wife.getName() +
            '}';
    }
}
```

- Wife.java



```
package org.cqipu.edu.bean;

public class Wife {
    private String name;
    private Husband husband;

    public void setName(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    public void setHusband(Husband husband) {
        this.husband = husband;
    }

    // toString()方法重写时需要注意: 不能直接输出husband, 输出husband.getName()。要不然会出现递归导致的栈内存溢出错误。
    @Override
    public String toString() {
        return "Wife{" +
            "name='" + name + '\'' +
            ", husband=" + husband.getName() +
            '}';
    }
}
```

- spring.xml



```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="husbandBean" class="org.cqipu.edu.bean.Husband" scope="singleton">
        <property name="name" value="张三"/>
        <property name="wife" ref="wifeBean"/>
    </bean>
    <bean id="wifeBean" class="org.cqipu.edu.bean.Wife" scope="singleton">
        <property name="name" value="小花"/>
        <property name="husband" ref="husbandBean"/>
    </bean>
</beans>
```

- 测试程序



```
package org.cqipu.edu.test;

import org.cqipu.edu.bean.Husband;
import org.cqipu.edu.bean.Wife;
import org.junit.Test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class CircularDependencyTest {

    @Test
    public void testSingletonAndSet(){
        ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring.xml");
        Husband husbandBean = applicationContext.getBean("husbandBean", Husband.class);
        Wife wifeBean = applicationContext.getBean("wifeBean", Wife.class);
        System.out.println(husbandBean);
        System.out.println(wifeBean);
    }
}
```

测试结果

测试 已通过: 1共 1 个测试

Husband{name='张三', wife=小花}
Wife{name='小花', husband=张三}

通过测试得知：在singleton + set注入的情况下，循环依赖是没有问题的。Spring可以解决这个问题。

prototype下的set注入产生的循环依赖

我们再来测试一下：prototype+set注入的方式下，循环依赖会不会出现问题？

- spring.xml



```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<beans xmlns="http://www.springframework.org/schema/beans"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="husbandBean" class="org.cqipu.edu.bean.Husband" scope="prototype">
        <property name="name" value="张三"/>
        <property name="wife" ref="wifeBean"/>
    </bean>
    <bean id="wifeBean" class="org.cqipu.edu.bean.Wife" scope="prototype">
        <property name="name" value="小花"/>
        <property name="husband" ref="husbandBean"/>
    </bean>
</beans>
```

测试结果

D:\Java21\bin\java.exe ...

org.springframework.beans.factory.BeanCreationException: Error creating bean with name 'husbandBean' defined in class path resource [spring.xml]: Cannot resolve reference to bean 'wifeBean' while setting bean property 'wife'

```
at org.springframework.beans.factory.support.BeanDefinitionValueResolver.resolveReference(BeanDefinitionValueResolver.java:377)
at org.springframework.beans.factory.support.BeanDefinitionValueResolver.resolveValueIfNecessary(BeanDefinitionValueResolver
.java:135)
at org.springframework.beans.factory.support.AbstractAutowireCapableBeanFactory.applyPropertyValues
(AbstractAutowireCapableBeanFactory.java:1682)
at org.springframework.beans.factory.support.AbstractAutowireCapableBeanFactory.populateBean(AbstractAutowireCapableBeanFactory
.java:1431)
at org.springframework.beans.factory.support.AbstractAutowireCapableBeanFactory.doCreateBean(AbstractAutowireCapableBeanFactory
.java:597)
at org.springframework.beans.factory.support.AbstractAutowireCapableBeanFactory.createBean(AbstractAutowireCapableBeanFactory
.java:520)
at org.springframework.beans.factory.support.AbstractBeanFactory.doGetBean(AbstractBeanFactory.java:344)
at org.springframework.beans.factory.support.AbstractBeanFactory.getBean(AbstractBeanFactory.java:205)
at org.springframework.context.support.AbstractApplicationContext.getBean(AbstractApplicationContext.java:1160)
> at org.cqipu.edu.test.CircularDependencyTest.testSingletonAndSet(CircularDependencyTest.java:14) <25 个内部行>
```

执行测试程序：发生了异常，异常信息如下：

```
Caused by: org.springframework.beans.factory.BeanCurrentlyInCreationException: Error creating bean with name
'husbandBean': Requested bean is currently in creation: Is there an unresolvable circular reference?
at org.springframework.beans.factory.support.AbstractBeanFactory.doGetBean(AbstractBeanFactory.java:265)
at org.springframework.beans.factory.support.AbstractBeanFactory.getBean(AbstractBeanFactory.java:199)
at
org.springframework.beans.factory.support.BeanDefinitionValueResolver.resolveReference(BeanDefinitionValueResolver.java:325)
... 44 more
```

翻译为：创建名为“husbandBean”的bean时出错：请求的bean当前正在创建中：是否存在无法解析的循环引用？

通过测试得知，当循环依赖的**所有Bean**的scope="prototype"的时候，产生的循环依赖，Spring是无法解决的，会出现**BeanCurrentlyInCreationException**异常。

大家可以测试一下，以上两个Bean，如果其中一个是singleton，另一个是prototype，是没有问题的。

为什么两个Bean都是prototype时会出错呢？

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans http://www.springframework.org/schema/beans-3.0.xsd">

    <bean id="husbandBean" class="com.powernode.spring6.bean.Husband" scope="prototype">
        <property name="name" value="张三"/>
        <property name="wife" ref="wifeBean"/>
    </bean>

    <bean id="wifeBean" class="com.powernode.spring6.bean.Wife" scope="prototype">
        <property name="name" value="小花"/>
        <property name="husband" ref="husbandBean"/>
    </bean>

</beans>
```

这是因为每一次注入的wifeBean必须是新new出来Bean导致的

这是因为每一次注入的husbandBean必须是新new出来的Bean导致的

老杜

singleton下的构造注入产生的循环依赖

我们再来测试一下singleton + 构造注入的方式下，spring是否能够解决这种循环依赖。

- Husband01.java

```
package org.cqipu.edu.bean;

public class Husband01 {
    private String name;
    private Wife wife;

    public Husband01(String name, Wife wife) {
        this.name = name;
        this.wife = wife;
    }

    // ----- 分割线 -----
    public String getName() {
        return name;
    }

    @Override
    public String toString() {
        return "Husband{" +
            "name='" + name + '\'' +
            ", wife=" + wife +
            '}';
    }
}
```

- Wife01.java

```
package org.cqipu.edu.bean;

public class Wife01 {
    private String name;
    private Husband husband;

    public Wife01(String name, Husband husband) {
        this.name = name;
    }
}
```

```

        this.husband = husband;
    }

    // -----分割线-----
    public String getName() {
        return name;
    }

    @Override
    public String toString() {
        return "Wife{" +
            "name='" + name + '\'' +
            ", husband=" + husband +
            '}';
    }
}

```

- spring2.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="hBean" class="org.cqipu.edu.bean.Husband01" scope="singleton">
        <constructor-arg name="name" value="张三"/>
        <constructor-arg name="wife" ref="wBean"/>
    </bean>

    <bean id="wBean" class="org.cqipu.edu.bean.Wife01" scope="singleton">
        <constructor-arg name="name" value="小花"/>
        <constructor-arg name="husband" ref="hBean"/>
    </bean>
</beans>

```

- 测试程序

```

@Test
public void testSingletonAndConstructor(){
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring2.xml");
    Husband hBean = applicationContext.getBean("hBean", Husband.class);
    Wife wBean = applicationContext.getBean("wBean", Wife.class);
    System.out.println(hBean);
    System.out.println(wBean);
}

```

 spring2.xml D:\java_Spring6_study\spring6-Circular-Dependency-Demo07\src\main\resources 6 个问题

- ❗ Bean 必须为 'org.cqipu.edu.bean.Wife' 类型 :7
- ❗ Bean 必须为 'org.cqipu.edu.bean.Husband' 类型 :12

```
D:\Java21\bin\java.exe ...
```

```
org.springframework.beans.factory.BeanCreationException: Error creating bean with name 'hBean' defined in class path resource
[spring2.xml]: Cannot resolve reference to bean 'wBean' while setting constructor argument

    at org.springframework.beans.factory.support.BeanDefinitionValueResolver.resolveReference(BeanDefinitionValueResolver.java:377)
    at org.springframework.beans.factory.support.BeanDefinitionValueResolver.resolveValueIfNecessary(BeanDefinitionValueResolver
.java:135)
    at org.springframework.beans.factory.support.ConstructorResolver.resolveConstructorArguments(ConstructorResolver.java:707)
    at org.springframework.beans.factory.support.ConstructorResolver.autowireConstructor(ConstructorResolver.java:211)
    at org.springframework.beans.factory.support.AbstractAutowireCapableBeanFactory.autowireConstructor
(AbstractAutowireCapableBeanFactory.java:1352)
    at org.springframework.beans.factory.support.AbstractAutowireCapableBeanFactory.createBeanInstance
(AbstractAutowireCapableBeanFactory.java:1189)
    at org.springframework.beans.factory.support.AbstractAutowireCapableBeanFactory.doCreateBean(AbstractAutowireCapableBeanFactory
.java:560)
    at org.springframework.beans.factory.support.AbstractAutowireCapableBeanFactory.createBean(AbstractAutowireCapableBeanFactory
.java:520)
```

执行结果：发生了异常，信息如下：

```
Caused by: org.springframework.beans.factory.BeanCurrentlyInCreationException: Error creating bean with name 'hBean':
Requested bean is currently in creation: Is there an unresolvable circular reference?
    at
    org.springframework.beans.factory.support.DefaultSingletonBeanRegistry.beforeSingletonCreation(DefaultSingletonBeanRegistry.j
ava:355)
    at org.springframework.beans.factory.support.DefaultSingletonBeanRegistry.getSingleton(DefaultSingletonBeanRegistry.java:227)
    at org.springframework.beans.factory.support.AbstractBeanFactory.doGetBean(AbstractBeanFactory.java:324)
    at org.springframework.beans.factory.support.AbstractBeanFactory.getBean(AbstractBeanFactory.java:199)
    at
    org.springframework.beans.factory.support.BeanDefinitionValueResolver.resolveReference(BeanDefinitionValueResolver.java:325)
    ... 56 more
```

和上一个测试结果相同，都是提示产生了循环依赖，并且Spring是无法解决这种循环依赖的。
为什么呢？

主要原因是通过构造方法注入导致的：因为构造方法注入会导致实例化对象的过程和对象属性赋值的过程没有分离开，必须在一起完成导致的。

Spring解决循环依赖的机理

结合之前学的Bean 生命周期前两步（第一步：实例化，第二步：属性赋值）来看。

- 什么是“三级缓存”？
 - 如果你去翻看 Spring 底层的源码，DefaultSingletonBeanRegistry 类，会发现这所谓的三级缓存，其实就是三个 Map 集合：
 - 一级缓存（singletonObjects）：单例池。存放的是完整的 Bean，经历了生命周期全过程，属性已经赋好值了。我们平时 getBean() 拿到的就是这里的对象。
 - 二级缓存（earlySingletonObjects）：早期对象池。存放的是半成品的 Bean，刚刚完成了生命周期第一步“实例化”，在内存里 new 出来了，但还没进行第二步“属性赋值”。
 - 三级缓存（singletonFactories）：单例工厂池。存放的是一个对象工厂，ObjectFactory，本质上是一个 Lambda 表达式，用来生成早期的半成品 Bean。

Spring 是如何利用三级缓存解开死循环的？

我们拿刚才的 Husband（丈夫）和 Wife（妻子）来一步步推演，来看 Spring 底层是怎么操作的：

- 实例化 Husband：Spring 准备创建 Husband，先调用无参构造，把 Husband 实例化（半成品）。
- 放入三级缓存：Spring 立刻把这个半成品的 Husband 包装成一个“对象工厂”，塞进三级缓存里。这就好比 Husband 刚出生，就在大喇叭里喊：“我在这呢，虽然我还没穿好衣服（没赋值），但如果谁急需我，可以先拿个我的引用去用！”

3. Husband 属性赋值：Spring 开始给 Husband 进行第二步操作（依赖注入），发现它需要注入 Wife。
4. 去拿 Wife：Spring 跑去一级、二级、三级缓存里找 Wife，发现都没有。于是 Spring 只能暂停 Husband 的创建，转头去创建 Wife。
5. 实例化 Wife 并放入三级缓存：同样的套路，Spring 把 Wife 实例化（半成品），也包装成工厂塞进三级缓存。
6. Wife 属性赋值（见证奇迹的时刻）：Spring 给 Wife 进行依赖注入，发现它需要 Husband。
7. Wife 去找 Husband：
 - 先去一级缓存找，没有（因为 Husband 还没走完流程）。
- 再去二级缓存找，没有。
- 最后去三级缓存找，找到了！因为在第 2 步的时候，Husband 已经把自己的工厂放进去了。
8. 拿到半成品 Husband：Wife 通过三级缓存里的工厂，拿到了半成品的 Husband 的引用。此时，Spring 会把半成品的 Husband 从三级缓存移到二级缓存。
9. Wife 创建完成：Wife 成功把半成品 Husband 注入到了自己的属性中。Wife 继续走完初始化流程，变成了一个完整的 Bean，最后被放入一级缓存，并清空它在二三级缓存里的记录。
10. Husband 恢复创建：Spring 回过头来，把刚才一级缓存里已经完全建好的 Wife，注入给 Husband。
11. Husband 创建完成：Husband 也走完剩下的流程，变成完整的 Bean，放入一级缓存。

Spring为什么可以解决set + singleton模式下循环依赖？

- 根本的原因在于：这种方式可以做到将“实例化Bean”和“给Bean属性赋值”这两个动作分开去完成。
- 实例化Bean的时候：调用无参数构造方法来完成。此时可以先不给属性赋值，可以提前将该Bean对象“曝光”给外界。
- 给Bean属性赋值的时候：调用setter方法来完成。
- 两个步骤是完全可以分离开去完成的，并且这两步不要求在同一个时间点上完成。

也就是说，Bean都是单例的，我们可以先把所有的单例Bean实例化出来，放到一个集合当中（我们可以称之为缓存），所有的单例Bean全部实例化完成之后，以后我们再慢慢的调用setter方法给属性赋值。这样就解决了循环依赖的问题。

那么在Spring框架底层源码级别上是如何实现的呢？请看：

```
5 usages 4 inheritors
public class DefaultSingletonBeanRegistry extends SimpleAliasRegistry implements SingletonBeanRegistry {

    Maximum number of suppressed exceptions to preserve.

    1 usage
    private static final int SUPPRESSED_EXCEPTIONS_LIMIT = 100;

    /** Cache of singleton objects: bean name to bean instance. */
    22 usages
    private final Map<String, Object> singletonObjects = new ConcurrentHashMap<>( initialCapacity: 256);

    /** Cache of singleton factories: bean name to ObjectFactory. */
    6 usages
    private final Map<String, ObjectFactory<?>> singletonFactories = new HashMap<>( initialCapacity: 16);

    /** Cache of early singleton objects: bean name to bean instance. */
    7 usages
    private final Map<String, Object> earlySingletonObjects = new ConcurrentHashMap<>( initialCapacity: 16);

    Set of registered singletons, containing the bean names in registration order.
```

在以上类中包含三个重要的属性：

- Cache of singleton objects: bean name to bean instance. 单例对象的缓存：key存储bean名称，value存储Bean对象【一级缓存】
- Cache of early singleton objects: bean name to bean instance. 早期单例对象的缓存：key存储bean名称，value存储早期的Bean对象【二级缓存】
- Cache of singleton factories: bean name to ObjectFactory. 单例工厂缓存：key存储bean名称，value存储该Bean对应的ObjectFactory对象【三级缓存】

这三个缓存其实本质上是三个Map集合。

我们再来看，在该类中有这样一个方法 `addSingletonFactory()`，这个方法的作用是：将创建Bean对象的ObjectFactory对象提前曝光。

Add the given singleton factory for building the specified singleton if necessary.
To be called for eager registration of singletons, e.g. to be able to resolve circular references.
Params: beanName – the name of the bean
singletonFactory – the factory for the singleton object

1 usage

```
protected void addSingletonFactory(String beanName, ObjectFactory<?> singletonFactory) {
    Assert.notNull(singletonFactory, message: "Singleton factory must not be null");
    synchronized (this.singletonObjects) {
        if (!this.singletonObjects.containsKey(beanName)) {
            this.singletonFactories.put(beanName, singletonFactory); 曝光该Bean对应的ObjectFactory对象
            this.earlySingletonObjects.remove(beanName);
            this.registeredSingletons.add(beanName);
        }
    }
}
```

老杜

在分析下面的代码

Params: beanName – the name of the bean to look for
allowEarlyReference – whether early references should be created or not
Returns: the registered singleton object, or null if none found

8 usages

@Nullable

```
protected Object getSingleton(String beanName, boolean allowEarlyReference) {
    // Quick check for existing instance without full singleton lock
    Object singletonObject = this.singletonObjects.get(beanName); 先从一级缓存中获取
    if (singletonObject == null && isSingletonCurrentlyInCreation(beanName)) {
        singletonObject = this.earlySingletonObjects.get(beanName); 一级缓存获取不到时，再从二级缓存中获取
        if (singletonObject == null && allowEarlyReference) {
            synchronized (this.singletonObjects) {
                // Consistent creation of early reference within full singleton lock
                singletonObject = this.singletonObjects.get(beanName);
                if (singletonObject == null) {
                    singletonObject = this.earlySingletonObjects.get(beanName);
                    if (singletonObject == null) { 二级缓存拿不到的话，从三级缓存中获取
                        ObjectFactory<?> singletonFactory = this.singletonFactories.get(beanName);
                        if (singletonFactory != null) { 三级缓存中存放了提前曝光的ObjectFactory对象。
                            singletonObject = singletonFactory.getObject();
                            this.earlySingletonObjects.put(beanName, singletonObject);
                            this.singletonFactories.remove(beanName);
                        }
                    }
                }
            }
        }
    }
    return singletonObject;
}
```

老杜

从源码中可以看到，spring会先从一级缓存中获取Bean，如果获取不到，则从二级缓存中获取Bean，如果二级缓存还是获取不到，则从三级缓存中获取之前曝光的ObjectFactory对象，通过ObjectFactory对象获取Bean实例，这样就解决了循环依赖的问题。

总结：

Spring只能解决setter方法注入的单例bean之间的循环依赖。ClassA依赖ClassB，ClassB又依赖ClassA，形成依赖闭环。Spring在创建ClassA对象后，不需要等给属性赋值，直接将其曝光到bean缓存当中。在解析ClassA的属性时，又发现依赖于ClassB，再次去获取ClassB，当解析ClassB的属性时，又发现需要ClassA的属性，但此时的ClassA已经被提前曝光加入了正在创建的bean的缓存中，则无需创建新的ClassA的实例，直接从缓存中获取即可。从而解决循环依赖问题。

回顾反射机制

分析方法四要素

我们先来看一下，不使用反射机制调用一个方法需要几个要素的参与。

有一个这样的类：

- SystemService

```
package org.cqipu.edu.reflect;
public class SystemService {

    public void logout(){
        System.out.println("退出系统");
    }

    public boolean login(String username, String password){
        if ("admin".equals(username) && "admin123".equals(password)) {
            return true;
        }
        return false;
    }
}
```

编写程序调用方法：

- ReflectTest01

```
package org.cqipu.edu.reflect;

public class ReflectTest01 {
    public static void main(String[] args) {

        // 创建对象
        SystemService systemService = new SystemService();

        // 调用方法并接收方法的返回值
        boolean success = systemService.login("admin", "admin123");

        System.out.println(success ? "登录成功" : "登录失败");
    }
}
```

执行结果

```
D:\Java21\bin\java.exe ...  
登录成功
```

通过以上第 16 行代码可以看出，调用一个方法，一般涉及到 4 个要素：

- 调用哪个对象的（systemService）
- 哪个方法（login）
- 传什么参数（"admin", "admin123"）
- 返回什么值（success）

获取Method

要使用反射机制调用一个方法，首先你要获取到这个方法。

在反射机制中Method实例代表的是一个方法。那么怎么获取Method实例呢？

有这样一个类：

- SystemService.java

```
package org.cqipu.edu.reflect;  
public class SystemService {  
  
    public void logout(){  
        System.out.println("退出系统");  
    }  
  
    public boolean login(String username, String password){  
        if ("admin".equals(username) && "admin123".equals(password)) {  
            return true;  
        }  
        return false;  
    }  
  
    public boolean login(String password){  
        if("110".equals(password)){  
            return true;  
        }  
        return false;  
    }  
}
```

我们如何获取到 logout ()、login (String,String)、login (String) 这三个方法呢？

要获取方法 Method，首先你需要获取这个类 Class。

- 获取类 Class 的代码

```
Class clazz = Class.forName("org.cqipu.edu.reflect.SystemService");
```

当拿到 Class 之后，调用 getDeclaredMethod () 方法可以获取到方法。

假如你要获取这个方法：`login(String username, String password)`

- 获取 login (String username, String password)



```
Method loginMethod = clazz.getDeclaredMethod("login", String.class, String.class);
```

假如你要获取到这个方法：`login(String password)`

- 获取 login (String password)



```
Method loginMethod = clazz.getDeclaredMethod("login", String.class);
```

获取一个方法，需要告诉 Java 程序，你要获取的方法的名字是什么，这个方法上每个形参的类型是什么。这样 Java 程序才能给你拿到对应的方法。

这样的设计也非常合理，因为在同一个类当中，方法是支持重载的，也就是说方法名可以一样，但参数列表一定是不一样的，所以获取一个方法需要提供方法名以及每个形参的类型。

假设有这样一个方法：



```
public void setAge(int age){  
    this.age = age;  
}
```

你要获取这个方法的话，代码应该这样写：

- 获取 setAge (int age)



```
Method setAgeMethod = clazz.getDeclaredMethod("setAge", int.class);
```

其中 setAge 是方法名，int.class 是形参的类型。

如果要获取上面的 logout 方法，代码应该这样写：

- 获取 logout ()



```
Method logoutMethod = clazz.getDeclaredMethod("Logout");
```

因为这个方法形式参数的个数是 0 个。所以只需要提供方法名就行了。你学会了吗？

调用Method

要让一个方法调用的话，就关联到四要素了：

- 调用哪个对象的
- 哪个方法
- 传什么参数
- 返回什么值
 - SystemService



```
package org.cqipu.edu.reflect;  
  
public class SystemService {  
  
    public void logout(){
```

```

        System.out.println("退出系统");
    }

    public boolean login(String username, String password){
        if ("admin".equals(username) && "admin123".equals(password)) {
            return true;
        }
        return false;
    }

    public boolean login(String password){
        if("110".equals(password)){
            return true;
        }
        return false;
    }
}

```

假如我们要调用的方法是：login (String, String)

- 第一步：创建对象（四要素之首：调用哪个对象的）

```

Class clazz = Class.forName("org.cqipu.edu.reflect.SystemService");
Object obj = clazz.newInstance();

```

- 第二步：获取方法 login (String,String)（四要素之一：哪个方法）

```

Method loginMethod = clazz.getDeclaredMethod("login", String.class, String.class);

```

- 第三步：调用方法

```

Object retValue = loginMethod.invoke(obj, "admin", "admin123");

```

解说四要素：

- 哪个对象：obj
- 哪个方法：loginMethod
- 传什么参数："admin", "admin123"
- 返回什么值：retValue

```

package org.cqipu.edu.reflect;

import java.lang.reflect.Method;

public class ReflectTest02 {
    public static void main(String[] args) throws Exception{
        Class clazz = Class.forName("org.cqipu.edu.reflect.SystemService");
        Object obj = clazz.newInstance();
        Method loginMethod = clazz.getDeclaredMethod("login", String.class, String.class);
        Object retValue = loginMethod.invoke(obj, "admin", "admin123");
        System.out.println(retValue);
    }
}

```

执行结果

```
D:\Java21\bin\java.exe ...  
true
```

那如果调用既没有参数，又没有返回值的logout方法，应该怎么做？

```
package org.cqipu.edu.reflect;  
import java.lang.reflect.Method;  
  
public class ReflectTest03 {  
    public static void main(String[] args) throws Exception{  
        Class clazz = Class.forName("org.cqipu.edu.reflect.SystemService");  
        Object obj = clazz.newInstance();  
        Method logoutMethod = clazz.getDeclaredMethod("Logout");  
        logoutMethod.invoke(obj);  
    }  
}
```

假设知道属性名

假设有这一个类

```
package org.cqipu.edu.reflect;  
public class User {  
    private String name;  
    private int age;  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    public int getAge() {  
        return age;  
    }  
  
    public void setAge(int age) {  
        this.age = age;  
    }  
  
    @Override  
    public String toString() {  
        return "User{" +  
            "name='" + name + '\'' +  
            ", age=" + age +  
            '}';  
    }  
}
```

你知道以下这几条信息：

- 类名是：com.powernode.reflect.User
- 该类中有 String 类型的 name 属性和 int 类型的 age 属性。
- 另外你也知道该类的设计符合 javabeen 规范。（也就是说属性私有化，对外提供 setter 和 getter 方法）

你如何通过反射机制给 User 对象的 name 属性赋值 zhangsan，给 age 属性赋值 20 岁。

```
package org.cqipu.edu.reflect;
import java.lang.reflect.Method;

public class UserTest {
    public static void main(String[] args) throws Exception{
        // 已知类名
        String className = "org.cqipu.edu.reflect.User";
        // 已知属性名
        String propertyName = "age";

        // 通过反射机制给User对象的age属性赋值20岁
        Class<?> clazz = Class.forName(className);
        Object obj = clazz.newInstance(); // 创建对象

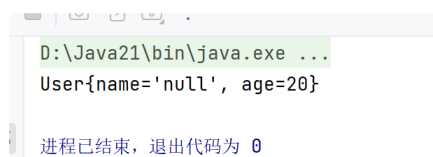
        // 根据属性名获取setter方法名
        String setMethodName = "set" + propertyName.toUpperCase().charAt(0) + propertyName.substring(1);

        // 获取Method
        Method setMethod = clazz.getDeclaredMethod(setMethodName, int.class);

        // 调用Method
        setMethod.invoke(obj, 20);

        System.out.println(obj);
    }
}
```

执行结果



```
D:\Java21\bin\java.exe ...
User{name='null', age=20}

进程已结束，退出代码为 0
```

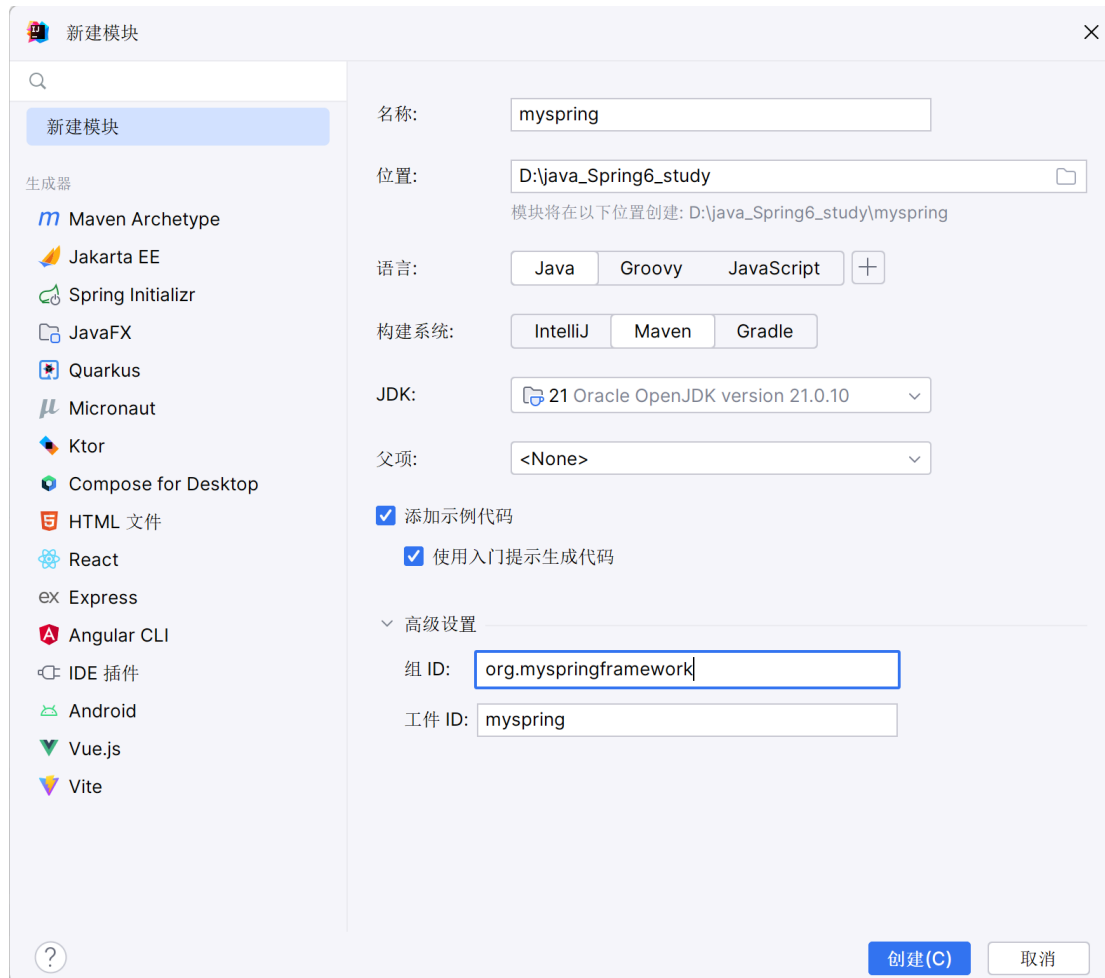
手写 Spring 框架

Spring IoC 容器的实现原理：工厂模式 + 解析 XML + 反射机制。

我们给自己的框架起名为：myspring（我的春天）

第一步：创建模块 myspring

采用 Maven 方式新建 Module：myspring



打包方式采用jar，并且引入dom4j和jaxen的依赖，因为要使用它解析XML文件，还有junit依赖。

- pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>org.myspringframework</groupId>
  <artifactId>myspring</artifactId>
  <version>1.0-SNAPSHOT</version>

  <dependencies>
    <dependency>
      <groupId>org.dom4j</groupId>
      <artifactId>dom4j</artifactId>
      <version>2.1.4</version>
    </dependency>
    <dependency>
      <groupId>jaxen</groupId>
      <artifactId>jaxen</artifactId>
      <version>1.2.0</version>
    </dependency>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.13.2</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

```

</dependencies>

<properties>
  <maven.compiler.source>21</maven.compiler.source>
  <maven.compiler.target>21</maven.compiler.target>
</properties>

</project>

```

第二步：准备好我们要管理的 Bean

准备好我们要管理的 Bean（这些 Bean 在将来开发完框架之后是要删除的）

注意包名，不要用 `org.springframework` 包，因为这些 Bean 不是框架内置的，是将来使用我们框架的程序员提供的。

```

package com.cqipu.myspring.bean;
public class Address {
    private String city;
    private String street;
    private String zipcode;

    public Address() {
    }

    public String getCity() {
        return city;
    }

    public void setCity(String city) {
        this.city = city;
    }

    public String getStreet() {
        return street;
    }

    public void setStreet(String street) {
        this.street = street;
    }

    public String getZipcode() {
        return zipcode;
    }

    public void setZipcode(String zipcode) {
        this.zipcode = zipcode;
    }

    @Override
    public String toString() {
        return "Address{" +
            "city='" + city + '\'' +
            ", street='" + street + '\'' +
            ", zipcode='" + zipcode + '\'' +
            '}';
    }
}

```

```

package com.cqipu.myspring.bean;

```



```

public class User {
    private String name;
    private int age;
    private Address addr;

    public User() {
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        this.age = age;
    }

    public Address getAddr() {
        return addr;
    }

    public void setAddr(Address addr) {
        this.addr = addr;
    }

    @Override
    public String toString() {
        return "User{" +
            "name='" + name + '\'' +
            ", age=" + age +
            ", addr=" + addr +
            '}';
    }
}

```

第三步：准备 myspring.xml 配置文件

将来在框架开发完毕之后，这个文件也是要删除的。因为这个配置文件的提供者应该是使用这个框架的程序员。

文件名随意，我们这里叫做：myspring.xml

文件放在类路径当中即可，我们这里把文件放到类的根路径下。

- myspring.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<beans>

    <bean id="userBean" class="com.cqipu.myspring.bean.User">
        <property name="name" value="张三"/>
        <property name="age" value="20"/>
        <property name="addr" ref="addrBean"/>
    </bean>

```

```
<bean id="addrBean" class="com.cqipu.myspring.bean.Address">
    <property name="city" value="北京"/>
    <property name="street" value="大兴区"/>
    <property name="zipcode" value="100001"/>
</bean>

</beans>
```

使用 value 给简单属性赋值。使用 ref 给非简单属性赋值。

第四步：编写 ApplicationContext 接口

ApplicationContext 接口中提供一个 getBean () 方法，通过该方法可以获取 Bean 对象。

注意包名：这个接口就是 myspring 框架中的一员了。

- ApplicationContext 接口

```
package org.myspringframework.core;
public interface ApplicationContext {

    /**
     * 根据bean的id获取bean实例。
     * @param beanId bean的id
     * @return bean实例
     */
    Object getBean(String beanId);
}
```

第五步：编写 ClassPathXmlApplicationContext

ClassPathXmlApplicationContext 是 ApplicationContext 接口的实现类。该类从类路径当中加载 myspring.xml 配置文件。

- ClassPathXmlApplicationContext

```
package org.myspringframework.core;

public class ClassPathXmlApplicationContext implements ApplicationContext{
    @Override
    public Object getBean(String beanId) {
        return null;
    }
}
```

第六步：确定采用 Map 集合存储 Bean

确定采用 Map 集合存储 Bean 实例。Map 集合的 key 存储 beanId，value 存储 Bean 实例。Map<String,Object>

在ClassPathXmlApplicationContext类中添加Map<String,Object>属性。

并且在ClassPathXmlApplicationContext类中添加构造方法，该构造方法的参数接收myspring.xml文件。

同时实现getBean方法。

- ClassPathXmlApplicationContext

```
package org.myspringframework.core;
```

```

import java.util.HashMap;
import java.util.Map;

public class ClassPathXmlApplicationContext implements ApplicationContext{
    /**
     * 存储bean的Map集合
     */
    private Map<String, Object> beanMap = new HashMap<>();

    /**
     * 在该构造方法中，解析myspring.xml文件，创建所有的Bean实例，并将Bean实例存放到Map集合中。
     * @param resource 配置文件路径（要求在类路径当中）
     */
    public ClassPathXmlApplicationContext(String resource) {

    }

    @Override
    public Object getBean(String beanId) {
        return beanMap.get(beanId);
    }
}

```

第七步：解析配置文件实例化所有 Bean

在 ClassPathXmlApplicationContext 的构造方法中解析配置文件，获取所有 bean 的类名，通过反射机制调用无参数构造方法创建 Bean。并且将 Bean 对象存放到 Map 集合中。

- ClassPathXmlApplicationContext构造方法

```

/**
 * 在该构造方法中，解析myspring.xml文件，创建所有的Bean实例，并将Bean实例存放到Map集合中。
 * @param resource 配置文件路径（要求在类路径当中）
 */
public ClassPathXmlApplicationContext(String resource) {
    try {
        SAXReader reader = new SAXReader();
        Document document = reader.read(ClassLoader.getSystemClassLoader().getResourceAsStream(resource));
        // 获取所有的bean标签
        List<Node> beanNodes = document.selectNodes("//bean");
        // 遍历集合
        beanNodes.forEach(beanNode -> {
            Element beanElt = (Element) beanNode;
            // 获取id
            String id = beanElt.attributeValue("id");
            // 获取className
            String className = beanElt.attributeValue("class");
            try {
                // 通过反射机制创建对象
                Class<?> clazz = Class.forName(className);
                Constructor<?> defaultConstructor = clazz.getDeclaredConstructor();
                Object bean = defaultConstructor.newInstance();
                // 存储到Map集合
                beanMap.put(id, bean);
            } catch (Exception e) {
                e.printStackTrace();
            }
        });
    } catch (Exception e) {
        e.printStackTrace();
    }
}

```

```
}
```

- ClassPathXmlApplicationContext---最终代码

```
package org.myspringframework.core;

import org.dom4j.Document;
import org.dom4j.Element;
import org.dom4j.Node;
import org.dom4j.io.SAXReader;
import java.lang.reflect.Constructor;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class ClassPathXmlApplicationContext implements ApplicationContext{
    /**
     * 存储bean的Map集合
     */
    private Map<String, Object> beanMap = new HashMap<>();

    /**
     * 在该构造方法中，解析myspring.xml文件，创建所有的Bean实例，并将Bean实例存放到Map集合中。
     * @param resource 配置文件路径（要求在类路径当中）
     */
    public ClassPathXmlApplicationContext(String resource) {
        try {
            // 创建dom4j解析器对象
            SAXReader reader = new SAXReader();
            // 读取类路径下的xml配置文件
            Document document = reader.read(ClassLoader.getSystemClassLoader().getResourceAsStream(resource));
            // 获取所有的bean标签
            List<Node> beanNodes = document.selectNodes("//bean");
            // 遍历所有bean标签
            beanNodes.forEach(beanNode -> {
                Element beanElt = (Element) beanNode;
                // 获取bean的id属性
                String id = beanElt.attributeValue("id");
                // 获取bean的class属性
                String className = beanElt.attributeValue("class");
                try {
                    // 通过反射机制创建对象（调用无参构造）
                    Class<?> clazz = Class.forName(className);
                    Constructor<?> defaultConstructor = clazz.getDeclaredConstructor();
                    Object bean = defaultConstructor.newInstance();
                    // 将bean对象存储到Map集合
                    beanMap.put(id, bean);
                } catch (Exception e) {
                    e.printStackTrace();
                }
            });
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    /**
     * 根据bean的id获取bean对象
     * @param beanId bean的id
     * @return bean实例
     */
    @Override
    public Object getBean(String beanId) {
```

```
        return beanMap.get(beanId);
    }
}
```

第八步：测试能否获取到Bean

编写测试程序。

测试程序

```
package com.cqipu.myspring.test;

import org.junit.Test;
import org.springframework.core.ApplicationContext;
import org.springframework.core.ClassPathXmlApplicationContext;

public class MySpringTest {
    @Test
    public void testMySpring(){
        ApplicationContext applicationContext = new ClassPathXmlApplicationContext("myspring.xml");
        Object userBean = applicationContext.getBean("userBean");
        Object addrBean = applicationContext.getBean("addrBean");
        System.out.println(userBean);
        System.out.println(addrBean);
    }
}
```

测试结果

✓ 测试 已通过: 1共 1 个测试 - 102毫秒

```
D:\Java21\bin\java.exe -ea -Didea.test.cyclic.buffer.size=1048576 "-javaagent:D:\JAVAidea\IntelliJ IDEA 2023.2\lib\idea_rt_
.jar=1451:D:\JAVAidea\IntelliJ IDEA 2023.2\bin" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8
-classpath "D:\JAVAidea\IntelliJ IDEA 2023.2\lib\idea_rt.jar;D:\JAVAidea\IntelliJ IDEA 2023.2\plugins\junit\lib\junit5-rt.jar;
D:\JAVAidea\IntelliJ IDEA 2023.2\plugins\junit\lib\junit-rt.jar;D:\java_Spring6_study\myspring\target\test-classes;
D:\java_Spring6_study\myspring\target\classes;C:\Users\jack\.m2\repository\org\dom4j\dom4j\2.1.4\dom4j-2.1.4.jar;C:\Users\jack\
.m2\repository\jaxen\jaxen\1.2.0\jaxen-1.2.0.jar;C:\Users\jack\.m2\repository\junit\junit\4.13.2\junit-4.13.2.jar;C:\Users\jack\
.m2\repository\org\hamcrest\hamcrest-core\1.3\hamcrest-core-1.3.jar" com.intellij.rt.junit.JUnit4Starter -ideVersion5 -junit4
com.cqipu.myspring.test.MySpringTest,testMySpring
User{name='null', age=0, addr=null}
Address{city='null', street='null', zipcode='null'}
```

通过测试 Bean 已经实例化成功了，属性的值是 null，这是我们能够想到的，毕竟我们调用的是无参数构造方法，所以属性都是默认值。

下一步就是我们应该如何给 Bean 的属性赋值呢？

第九步：给 Bean 的属性赋值

通过反射机制调用 set 方法，给 Bean 的属性赋值。

继续在 ClassPathXmlApplicationContext 构造方法中编写代码。

```
package org.springframework.core;
import org.dom4j.Document;
import org.dom4j.Element;
import org.dom4j.Node;
import org.dom4j.io.SAXReader;
import java.lang.reflect.Constructor;
import java.lang.reflect.Method;
```

```

import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class ClassPathXmlApplicationContext implements ApplicationContext{
    /**
     * 存储bean的Map集合
     */
    private Map<String, Object> beanMap = new HashMap<>();

    /**
     * 在该构造方法中，解析myspring.xml文件，创建所有的Bean实例，并将Bean实例存放到Map集合中。
     * @param resource 配置文件路径（要求在类路径当中）
     */
    public ClassPathXmlApplicationContext(String resource) {
        try {
            SAXReader reader = new SAXReader();
            Document document = reader.read(ClassLoader.getSystemClassLoader().getResourceAsStream(resource));
            // 获取所有的bean标签
            List<Node> beanNodes = document.selectNodes("//bean");
            // 遍历集合（这里的遍历只实例化Bean，不给属性赋值。为什么要这样做？）
            beanNodes.forEach(beanNode -> {
                Element beanElt = (Element) beanNode;
                // 获取id
                String id = beanElt.attributeValue("id");
                // 获取className
                String className = beanElt.attributeValue("class");
                try {
                    // 通过反射机制创建对象
                    Class<?> clazz = Class.forName(className);
                    Constructor<?> defaultConstructor = clazz.getDeclaredConstructor();
                    Object bean = defaultConstructor.newInstance();
                    // 存储到Map集合
                    beanMap.put(id, bean);
                } catch (Exception e) {
                    e.printStackTrace();
                }
            });
            // 再重新遍历集合，这次遍历是为了给Bean的所有属性赋值。
            // 思考：为什么不在上面的循环中给Bean的属性赋值，而在这里再重新遍历一次呢？
            // 通过这里你是否能够想到Spring是如何解决循环依赖的：实例化和属性赋值分开。
            beanNodes.forEach(beanNode -> {
                Element beanElt = (Element) beanNode;
                // 获取bean的id
                String beanId = beanElt.attributeValue("id");
                // 获取所有property标签
                List<Element> propertyElts = beanElt.elements("property");
                // 遍历所有属性
                propertyElts.forEach(propertyElt -> {
                    try {
                        // 获取属性名
                        String propertyName = propertyElt.attributeValue("name");
                        // 获取属性类型
                        Class<?> propertyType =
                            beanMap.get(beanId).getClass().getDeclaredField(propertyName).getType();
                        // 获取set方法名
                        String setMethodName = "set" + propertyName.toUpperCase().charAt(0) +
                            propertyName.substring(1);
                        // 获取set方法
                        Method setMethod = beanMap.get(beanId).getClass().getDeclaredMethod(setMethodName,
                            propertyType);

                        // 获取属性的值，值可能是value，也可能是ref。
                        // 获取value
                        String propertyValue = propertyElt.attributeValue("value");
                        // 获取ref

```

```

        String propertyRef = propertyElt.attributeValue("ref");
        Object propertyVal = null;
        if (propertyValue != null) {
            // 该属性是简单属性
            String propertyTypeSimpleName = propertyType.getSimpleName();
            switch (propertyTypeSimpleName) {
                case "byte": case "Byte":
                    propertyVal = Byte.valueOf(propertyValue);
                    break;
                case "short": case "Short":
                    propertyVal = Short.valueOf(propertyValue);
                    break;
                case "int": case "Integer":
                    propertyVal = Integer.valueOf(propertyValue);
                    break;
                case "Long": case "Long":
                    propertyVal = Long.valueOf(propertyValue);
                    break;
                case "float": case "Float":
                    propertyVal = Float.valueOf(propertyValue);
                    break;
                case "double": case "Double":
                    propertyVal = Double.valueOf(propertyValue);
                    break;
                case "boolean": case "Boolean":
                    propertyVal = Boolean.valueOf(propertyValue);
                    break;
                case "char": case "Character":
                    propertyVal = propertyValue.charAt(0);
                    break;
                case "String":
                    propertyVal = propertyValue;
                    break;
            }
            setMethod.invoke(beanMap.get(beanId), propertyVal);
        }
        if (propertyRef != null) {
            // 该属性不是简单属性
            setMethod.invoke(beanMap.get(beanId), beanMap.get(propertyRef));
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}
});
});

} catch (Exception e) {
    e.printStackTrace();
}
}

@Override
public Object getBean(String beanId) {
    return beanMap.get(beanId);
}
}

```

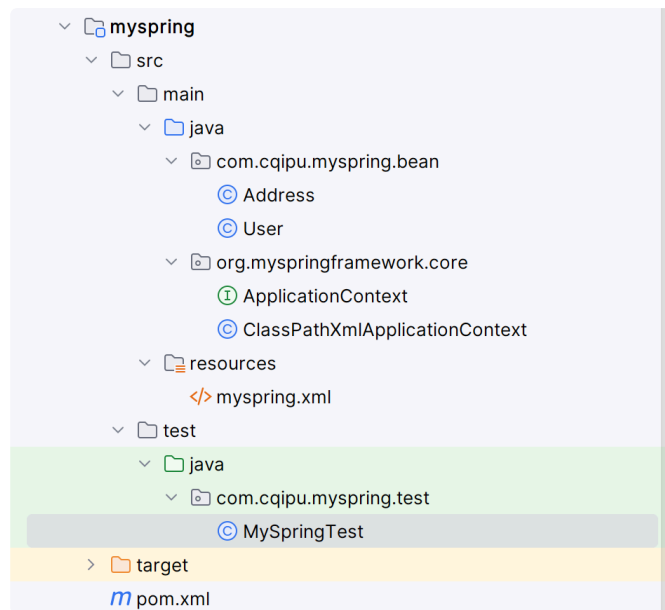
重点处理：当 property 标签中是 value 怎么办？是 ref 怎么办？

执行测试程序：

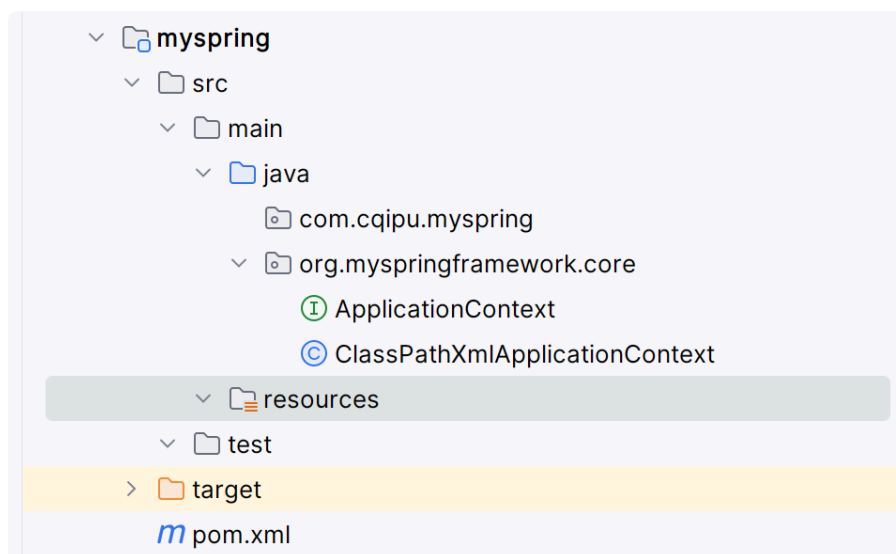
```
D:\Java21\bin\java.exe -ea -Didea.test.cyclic.buffer.size=1048576 "-javaagent:D:\JAVAidea\IntelliJ IDEA 2023.2\lib\idea_rt.jar=10877:D:\JAVAidea\IntelliJ IDEA 2023.2\bin" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath "D:\JAVAidea\IntelliJ IDEA 2023.2\lib\idea_rt.jar;D:\JAVAidea\IntelliJ IDEA 2023.2\plugins\junit\lib\junit5-rt.jar;D:\JAVAidea\IntelliJ IDEA 2023.2\plugins\junit\lib\junit-rt.jar;D:\java_Spring6_study\myspring\target\test-classes;D:\java_Spring6_study\myspring\target\classes;C:\Users\jack\.m2\repository\org\dom4j\dom4j\2.1.4\dom4j-2.1.4.jar;C:\Users\jack\.m2\repository\jaxen\jaxen\1.2.0\jaxen-1.2.0.jar;C:\Users\jack\.m2\repository\junit\junit\4.13.2\junit-4.13.2.jar;C:\Users\jack\.m2\repository\org\hamcrest\hamcrest-core\1.3\hamcrest-core-1.3.jar" com.intellij.rt.junit.JUnit4Starter -ideVersion5 -junit4 com.cqipu.myspring.test.MySpringTest, testMySpring
User{name='张三', age=20, addr=Address{city='北京', street='大兴区', zipcode='1000001'}}
Address{city='北京', street='大兴区', zipcode='1000001'}
```

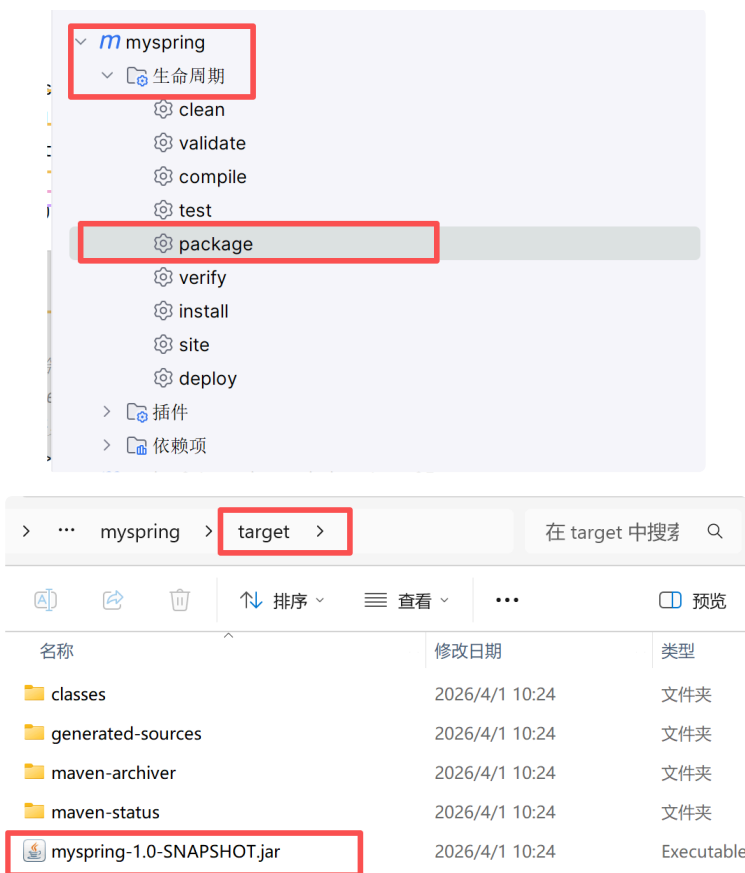
第十步：打包分布

将多余的类以及配置文件删除，使用maven打包分布



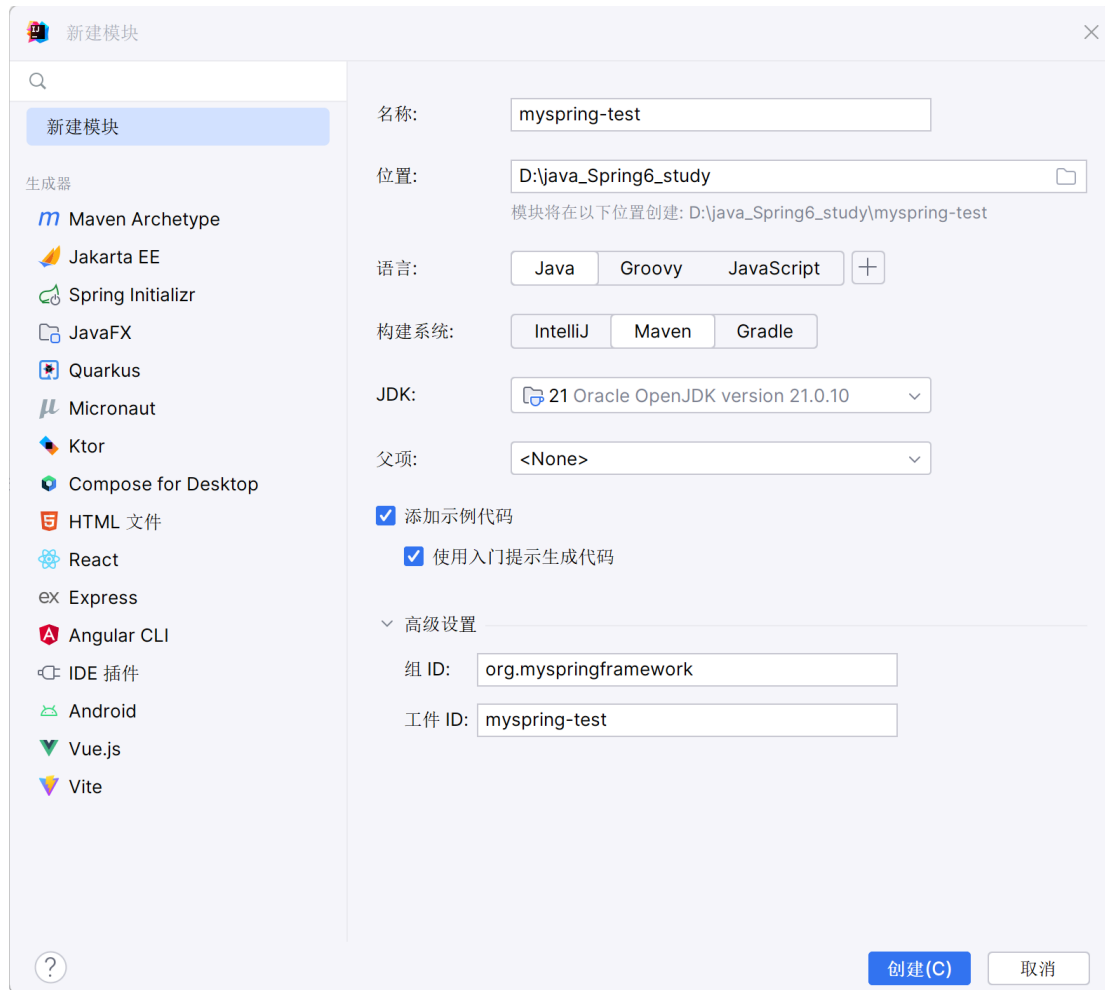
- 删除多余文件
 - src/test/java 文件夹
 - src/main/java/com.cqipu.myspring.bean
 - src/main/resources/myspring.xml





第十一步：站在程序员角度使用myspring框架

新建模块：myspring-test



引入myspring 框架依赖：

- pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>org.myspringframework</groupId>
    <artifactId>myspring-test</artifactId>
    <version>1.0-SNAPSHOT</version>

    <dependencies>
        <dependency>
            <groupId>org.myspringframework</groupId>
            <artifactId>myspring</artifactId>
            <version>1.0-SNAPSHOT</version>
        </dependency>
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>4.13.2</version>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <properties>
        <maven.compiler.source>21</maven.compiler.source>
        <maven.compiler.target>21</maven.compiler.target>
    </properties>
</project>
```

```
</properties>
</project>
```

编写Bean

- UserDao

```
package com.cqipu.myspring.bean;
public class UserDao {
    public void insert(){
        System.out.println("UserDao 正在插入数据");
    }
}
```

- UserService

```
package com.cqipu.myspring.bean;

public class UserService {
    private UserDao userDao;

    public void setUserDao(UserDao userDao) {
        this.userDao = userDao;
    }

    public void save(){
        System.out.println("UserService 开始执行save操作");
        userDao.insert();
        System.out.println("UserService 执行save操作结束");
    }
}
```

编写myspring.xml 文件

- myspring.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<beans>

    <bean id="userServiceBean" class="com.cqipu.myspring.bean.UserService">
        <property name="userDao" ref="userDaoBean"/>
    </bean>

    <bean id="userDaoBean" class="com.cqipu.myspring.bean.UserDao"/>

</beans>
```

编写测试程序

```
package com.cqipu.myspring.test;

import com.cqipu.myspring.bean.UserService;
import org.junit.Test;
import org.springframework.core.ApplicationContext;
import org.springframework.core.ClassPathXmlApplicationContext;

public class MySpringTest {
```

```

@Test
public void testMySpring(){
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("myspring.xml");
    UserService userServiceBean = (UserService) applicationContext.getBean("userServiceBean");
    userServiceBean.save();
}
}

```

测试结果：

UserService开始执行save操作
 UserDao正在插入数据
 UserService执行save操作结束

Spring IoC 注解式开发

回顾注解

注解的存在主要是为了简化 XML 的配置。Spring6 倡导全注解开发。
 我们来回顾一下：

1. 第一：注解怎么定义，注解中的属性怎么定义？
2. 第二：注解怎么使用？
3. 第三：通过反射机制怎么读取注解？

注解怎么定义，注解中的属性怎么定义？

- 自定义注解

```

package org.cqipu.edu.annotation;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Target(value = {ElementType.TYPE})
@Retention(value = RetentionPolicy.RUNTIME)
public @interface Component {
    String value();
}

```

以上是自定义了一个注解：Component

该注解上面修饰的注解包括：Target 注解和 Retention 注解，这两个注解被称为元注解。

1. Target 注解用来设置 Component 注解可以出现的位置，以上代表表示 Component 注解只能用在类和接口上。
2. Retention 注解用来设置 Component 注解的保持性策略，以上代表 Component 注解可以被反射机制读取。
3. String value (); 是 Component 注解中的一个属性。该属性类型 String，属性名是 value。

注解怎么使用？

- User



```
package org.cqipu.edu.bean;

import org.cqipu.edu.annotation.Component;

@Component(value = "userBean")
public class User {
}
```

- 用法简单，语法格式：@注解类型名 (属性名 = 属性值, 属性名 = 属性值, 属性名 = 属性值.....)
- userBean 为什么使用双引号括起来，因为 value 属性是 String 类型，字符串。
- 另外如果属性名是 value，则在使用的时候可以省略属性名，例如：



```
package org.cqipu.edu.bean;

import org.cqipu.edu.annotation.Component;

//@Component(value = "userBean")
@Component("userBean")
public class User {
}
```

通过反射机制怎么读取注解？

接下来，我们来写一段程序，当 Bean 类上有 Component 注解时，则实例化 Bean 对象，如果没有，则不实例化对象。

我们准备两个 Bean，一个上面有注解，一个上面没有注解。

- 有注解的Bean
 - User



```
package org.cqipu.edu.bean;

import org.cqipu.edu.annotation.Component;

@Component("userBean")
public class User {
}
```

- 没有注解的Bean
 - Vip



```
package org.cqipu.edu.bean;

public class Vip {
}
```

假设我们现在只知道包名：org.cqipu.edu.bean。至于这个包下有多少个Bean我们不知道。哪些Bean上有注解，哪些Bean上没有注解，这些我们都不知道，如何通过程序全自动化判断。

- 反射解析注解



```
package org.cqipu.edu.test;

import org.cqipu.edu.annotation.Component;
```

```

import java.io.File;
import java.net.URL;
import java.util.Arrays;
import java.util.Enumeration;
import java.util.HashMap;
import java.util.Map;

public class Test {
    public static void main(String[] args) throws Exception {
        // 存放Bean的Map集合。key存储beanId。value存储Bean。
        Map<String, Object> beanMap = new HashMap<>();

        String packageName = "org.cqipu.edu.bean";
        String path = packageName.replaceAll("\\.", "/");
        URL url = ClassLoader.getSystemClassLoader().getResource(path);
        File file = new File(url.getPath());
        File[] files = file.listFiles();
        Arrays.stream(files).forEach(f -> {
            String className = packageName + "." + f.getName().split("\\.")[0];
            try {
                Class<?> clazz = Class.forName(className);
                if (clazz.isAnnotationPresent(Component.class)) {
                    Component component = clazz.getAnnotation(Component.class);
                    String beanId = component.value();
                    Object bean = clazz.newInstance();
                    beanMap.put(beanId, bean);
                }
            } catch (Exception e) {
                e.printStackTrace();
            }
        });

        System.out.println(beanMap);
    }
}

```

测试结果

```

D:\Java21\bin\java.exe "-javaagent:D:\JAVAidea\IntelliJ IDEA 2023.2\lib\idea_rt.jar=10615:D:\JAVAidea\IntelliJ IDEA 2023.2\bin" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath D:\java_Spring6_study\spring6-ioc-annotation-demo09\target\test-classes;
D:\java_Spring6_study\spring6-ioc-annotation-demo09\target\classes org.cqipu.edu.test.Test
{userBean=org.cqipu.edu.bean.User@3d646c37}

```

声明 Bean 的注解

负责声明 Bean 的注解，常见的包括四个：

1. @Component
2. @Controller
3. @Service
4. @Repository

源码如下：

- @Component注解



```
package org.cqipu.edu.annotation;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Target(value = {ElementType.TYPE})
@Retention(value = RetentionPolicy.RUNTIME)
public @interface Component {
    String value() default "";
}
```

- @Controller注解



```
package org.cqipu.edu.stereotype;

import java.lang.annotation.Documented;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

import org.cqipu.edu.annotation.Component;
import org.springframework.core.annotation.AliasFor;

@Target({ElementType.TYPE})
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Component
public @interface Controller {
    @AliasFor(
        annotation = Component.class
    )
    String value() default "";
}
```

- @Service注解



```
package org.cqipu.edu.stereotype;
import java.lang.annotation.Documented;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

import org.cqipu.edu.annotation.Component;
import org.springframework.core.annotation.AliasFor;

@Target({ElementType.TYPE})
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Component
public @interface Service {
    @AliasFor(
        annotation = Component.class
    )
    String value() default "";
}
```

- @Repository注解

```

package org.cqipu.edu.stereotype;
import java.lang.annotation.Documented;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

import org.cqipu.edu.annotation.Component;
import org.springframework.core.annotation.AliasFor;

@Target({ElementType.TYPE})
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Component
public @interface Repository {
    @AliasFor(
        annotation = Component.class
    )
    String value() default "";
}

```

通过源码可以看到，@Controller、@Service、@Repository 这三个注解都是 @Component 注解的别名。

也就是说：这四个注解的功能都一样。用哪个都可以。

只是为了增强程序的可读性，建议：

- 控制器类上使用：Controller
- service 类上使用：Service
- dao 类上使用：Repository

他们都是只有一个 value 属性。value 属性用来指定 bean 的 id，也就是 bean 的名字。

```
<bean id="userBean" class="com.powernode.spring6.bean.User"/>
```

value属性指定的就是bean的id

```

package com.powernode.spring6.bean;

import org.springframework.stereotype.Component;

@Component(value = "userBean")
public class User {
}

```

老杜

Spring 注解的使用

如何使用以上的注解呢？

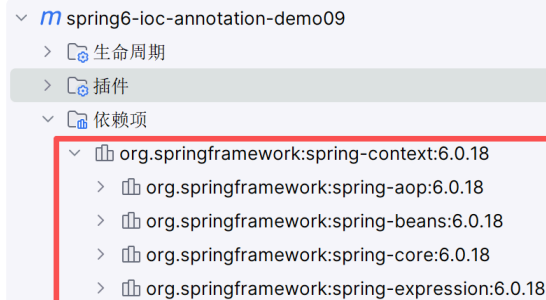
- 第一步：加入 aop 的依赖
- 第二步：在配置文件中添加 context 命名空间
- 第三步：在配置文件中指定扫描的包
- 第四步：在 Bean 类上使用注解

第一步：加入 aop 的依赖

我们可以看到当加入 spring-context 依赖之后，会关联加入 aop 的依赖。所以这一步不用做。

- pom.xml

```
<dependencies>
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>6.0.18</version>
  </dependency>
</dependencies>
```



```
spring6-ioc-annotation-demo09
├── 生命周期
├── 插件
└── 依赖项
    ├── org.springframework:spring-context:6.0.18
    │   ├── org.springframework:spring-aop:6.0.18
    │   ├── org.springframework:spring-beans:6.0.18
    │   ├── org.springframework:spring-core:6.0.18
    │   └── org.springframework:spring-expression:6.0.18
```

第二步：在配置文件中添加context命名空间

- spring.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/context
                           http://www.springframework.org/schema/context/spring-context.xsd">

</beans>
```

第三步：在配置文件中指定要扫描的包

- spring.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/context
                           http://www.springframework.org/schema/context/spring-context.xsd">
  <context:component-scan base-package="org.cqipu.edu.bean"/>
</beans>
```

第四步：在Bean类上使用注解

- User

```

package org.cqipu.edu.bean;
import org.springframework.stereotype.Component;

@Component(value = "userBean")
public class User {
}

```

测试程序

```

package org.cqipu.edu.test;

import org.cqipu.edu.bean.User;
import org.junit.Test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class AnnotationTest {
    @Test
    public void testBean(){
        ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring.xml");
        User userBean = applicationContext.getBean("userBean", User.class);
        System.out.println(userBean);
    }
}

```

测试结果

```

✓ 测试 已通过: 1共 1 个测试 - 372毫秒
D:\Java21\bin\java.exe ...
org.cqipu.edu.bean.User@47987356

```

如果注解的属性名是value，那么value是可以省略的

- Vip

```

package org.cqipu.edu.bean;
import org.springframework.stereotype.Component;

@Component("vipBean")
public class Vip {
}

```

测试程序

```

import org.cqipu.edu.bean.Vip;
@Test
public void testBean01(){
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring.xml");
    Vip vipBean = applicationContext.getBean("vipBean", Vip.class);
    System.out.println(vipBean);
}

```

```

D:\Java21\bin\java.exe ...
org.cqipu.edu.bean.Vip@6283d8b8

```

如果把 value 属性彻底去掉，spring 会被 Bean 自动取名吗？会的。并且默认名字的规律是：Bean 类名首字母小写即可。

- BankDao

```
package org.cqipu.edu.bean;
import org.springframework.stereotype.Component;

@Component
public class BankDao {
}
```

也就是说，这个 BankDao 的 bean 的名字为：bankDao

测试一下

```
package org.cqipu.edu.test;

import org.cqipu.edu.bean.BankDao;
import org.junit.Test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class AnnotationTest {
    @Test
    public void testBean02(){
        ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring.xml");
        BankDao bankDao = applicationContext.getBean("bankDao", BankDao.class);
        System.out.println(bankDao);
    }
}
```

✓ 测试 已通过: 1 共 1 个测试 - 323毫秒

D:\Java21\bin\java.exe ...
org.cqipu.edu.bean.BankDao@611889f4

我们将Component注解换成其它三个注解，看看是否可以用：

- 换成@Controller注解

```
package org.cqipu.edu.bean;

import org.springframework.stereotype.Controller;

@Controller
public class BankDao {
}
```

测试结果

🔍 🔄 📄 🗑️ :

✓ 测试 已通过: 1 共 1 个测试 - 323毫秒

D:\Java21\bin\java.exe ...
org.cqipu.edu.bean.BankDao@4738a206

剩下的两个注解大家可以测试一下。

如果是多个包怎么办？有两种解决方案：

- 第一种：在配置文件中指定多个包，用逗号隔开。
- 第二种：指定多个包的共同父包。

先来测试一下逗号（英文）的方式：

创建一个新的包：bean2，定义一个Bean类。

- bean2包下的Order

```
package org.cqipu.edu.bean2;

import org.springframework.stereotype.Service;

@Service
public class Order {
}
```

配置文件修改

- spring.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/context
                           http://www.springframework.org/schema/context/spring-context.xsd">
    <context:component-scan base-package="org.cqipu.edu.bean,org.cqipu.edu.bean2"/>
</beans>
```

测试程序

```
package org.cqipu.edu.test;
import org.cqipu.edu.bean.BankDao;
import org.cqipu.edu.bean2.Order;
import org.junit.Test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class AnnotationTest {
    @Test
    public void testBean03(){
        ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring.xml");
        BankDao bankDao = applicationContext.getBean("bankDao", BankDao.class);
        System.out.println(bankDao);
        Order order = applicationContext.getBean("order", Order.class);
        System.out.println(order);
    }
}
```

测试结果

✓ 测试 已通过: 1共 1 个测试 - 631毫秒

```
D:\Java21\bin\java.exe ...
org.cqipu.edu.bean.BankDao@58695725
org.cqipu.edu.bean2.Order@543588e6
```

我们再来看看，指定共同的父包行不行：

- spring.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/context
                           http://www.springframework.org/schema/context/spring-context.xsd">
<!--      <context:component-scan base-package="org.cqipu.edu.bean,org.cqipu.edu.bean2"/>-->
    <context:component-scan base-package="org.cqipu.edu"/>
</beans>
```

执行刚刚的测试程序

```
✓ 测试 已通过: 1共 1 个测试 - 390毫秒
D:\Java21\bin\java.exe ...
org.cqipu.edu.bean.BankDao@c88a337
org.cqipu.edu.bean2.Order@5d0a1059
```

选择性实例化 Bean

假设在某个包下有很多 Bean，有的 Bean 上标注了 Component，有的标注了 Controller，有的标注了 Service，有的标注了 Repository，现在由于某种特殊业务的需要，只允许其中所有的 Controller 参与 Bean 管理，其他的都不实例化。这应该怎么办呢？

- 这里为了方便，将这几个类都定义到同一个 java 源文件中了
 - A.java

```
package org.cqipu.edu.bean3;

import org.springframework.stereotype.Component;
import org.springframework.stereotype.Controller;
import org.springframework.stereotype.Repository;
import org.springframework.stereotype.Service;

@Component
public class A {
    public A() {
        System.out.println("A的无参数构造方法执行");
    }
}

@Controller
class B {
    public B() {
        System.out.println("B的无参数构造方法执行");
    }
}

@Service
class C {
    public C() {
        System.out.println("C的无参数构造方法执行");
    }
}

@Repository
class D {
    public D() {
        System.out.println("D的无参数构造方法执行");
    }
}
```

```

    }
}

@Controller
class E {
    public E() {
        System.out.println("E的无参数构造方法执行");
    }
}

@Controller
class F {
    public F() {
        System.out.println("F的无参数构造方法执行");
    }
}

```

- 我只想实例化bean3包下的Controller。配置文件这样写：
 - spring-choose.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/beans/spring-beans.xsd
                           http://www.springframework.org/schema/context
                           http://www.springframework.org/schema/context/spring-context.xsd">

    <context:component-scan base-package="org.cqipu.edu.bean3" use-default-filters="false">
        <context:include-filter type="annotation" expression="org.springframework.stereotype.Controller"/>
    </context:component-scan>
</beans>

```

1. use-default-filters="true" 表示：使用 spring 默认的规则，只要有 Component、Controller、Service、Repository 中的任意一个注解标注，则进行实例化。
2. use-default-filters="false" 表示：不再 spring 默认实例化规则，即使有 Component、Controller、Service、Repository 这些注解标注，也不再实例化。
3. <context:include-filter type="annotation" expression="org.springframework.stereotype.Controller"/> 表示只有 Controller 进行实例化。

测试程序

```

@Test
public void testChoose(){
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring-choose.xml");
}

```

✓ 测试 已通过: 1共 1 个测试 - 261毫秒

```

D:\Java21\bin\java.exe ...
B的无参数构造方法执行
E的无参数构造方法执行
F的无参数构造方法执行

```

也可以将use-default-filters设置为true（不写就是true），并且采用exclude-filter方式排出哪些注解标注的Bean不参与实例化：

- spring-choose.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/context
                           http://www.springframework.org/schema/context/spring-context.xsd">

    <!-- <context:component-scan base-package="org.cqipu.edu.bean3" use-default-filters="false">-->
    <!-- <context:include-filter type="annotation" expression="org.springframework.stereotype.Controller"/>-->
    <!-- </context:component-scan>-->

    <context:component-scan base-package="org.cqipu.edu.bean3">
        <context:exclude-filter type="annotation" expression="org.springframework.stereotype.Repository"/>
        <context:exclude-filter type="annotation" expression="org.springframework.stereotype.Service"/>
        <context:exclude-filter type="annotation" expression="org.springframework.stereotype.Controller"/>
    </context:component-scan>

</beans>
```

测试程序：

✓ 测试 已通过: 1共 1 个测试 - 346毫秒

D:\Java21\bin\java.exe ...

A的无参数构造方法执行

负责注入的注解

@Component @Controller @Service @Repository 这四个注解是用来声明 Bean 的，声明后这些 Bean 将被实例化。接下来我们看一下，如何给 Bean 的属性赋值。给 Bean 属性赋值需要用到这些注解：

- @Value
- @Autowired
- @Qualifier
- @Resource

@Value

当属性的类型是简单类型时，可以使用 @Value 注解进行注入。

```
package org.cqipu.edu.bean4;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

@Component
public class User {
    @Value(value = "zhangsan")
    private String name;
    @Value("20")
    private int age;

    @Override
    public String toString() {
        return "User{" +
            "name='" + name + '\'' +
            ", age=" + age +
            '}';
    }
}
```

```
}
```

- 开启包扫描：
 - spring-injection.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/context
                           http://www.springframework.org/schema/context/spring-context.xsd"
       >
    <context:component-scan base-package="org.cqipu.edu.bean4"/>
</beans>
```

- 测试程序

```
@Test
public void testValue(){
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring-injection.xml");
    Object user = applicationContext.getBean("user");
    System.out.println(user);
}
```

✓ 测试 已通过: 1共 1 个测试 - 330毫秒

D:\Java21\bin\java.exe ...

User{name='zhangsan', age=20}

通过以上代码可以发现，我们并没有给属性提供setter方法，但仍然可以完成属性赋值。

如果提供setter方法，并且在setter方法上添加@Value注解，可以完成注入吗？尝试一下：

- User

```
package org.cqipu.edu.bean4;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

@Component
public class User {
    private String name;
    private int age;
    @Value("李四")
    public void setName(String name) {
        this.name = name;
    }

    @Value("30")
    public void setAge(int age) {
        this.age = age;
    }

    @Override
    public String toString() {
```



```

        return "User{" +
            "name='" + name + '\'' +
            ", age=" + age +
            '}';
    }
}

```

执行结果

✓ 测试 已通过: 1共 1 个测试 - 269毫秒

```

D:\Java21\bin\java.exe ...
User{name='李四', age=30}

```

通过测试可以得知，@Value注解可以直接使用在属性上，也可以使用在setter方法上。都是可以的。都可以完成属性的赋值。

为了简化代码，以后我们一般不提供setter方法，直接在属性上使用@Value注解完成属性赋值。

出于好奇，我们再来测试一下，是否能够通过构造方法完成注入：

- User

```

package org.cqipu.edu.bean4;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

@Component
public class User {

    private String name;

    private int age;

    public User(@Value("隔壁老王") String name, @Value("33") int age) {
        this.name = name;
        this.age = age;
    }

    @Override
    public String toString() {
        return "User{" +
            "name='" + name + '\'' +
            ", age=" + age +
            '}';
    }
}

```

✓ 测试 已通过: 1共 1 个测试 - 263毫秒

```

D:\Java21\bin\java.exe ...
User{name='隔壁老王', age=33}

```

通过测试得知：@Value 注解可以出现在属性上、setter 方法上、以及构造方法的形参上。可见 Spring 给我们提供了多样化的注入。太灵活了。

@Autowired 与 @Qualifier

@Autowired 注解可以用来注入非简单类型。被翻译为：自动连线的，或者自动装配。

单独使用 @Autowired 注解，默认根据类型装配。【默认是 byType】

看一下它的源码：

- @Autowired 源码

```
package org.springframework.beans.factory.annotation;

import java.lang.annotation.Documented;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Target({ElementType.CONSTRUCTOR, ElementType.METHOD, ElementType.PARAMETER, ElementType.FIELD,
ElementType.ANNOTATION_TYPE})
@Retention(RetentionPolicy.RUNTIME)
@Documented
public @interface Autowired {
    boolean required() default true;
}
```

源码中有两处需要注意：

- 第一处：该注解可以标注在哪里？
 - 构造方法上
 - 方法上
 - 形参上
 - 属性上
 - 注解上
- 第二处：该注解有一个 required 属性，默认值是 true，表示在注入的时候要求被注入的 Bean 必须是存在的，如果不存在则报错。如果 required 属性设置为 false，表示注入的 Bean 存在或者不存在都没关系，存在的话就注入，不存在的话，也不报错。

我们先在属性上使用 @Autowired 注解：

- UserDao接口

```
package org.cqipu.edu.dao;

public interface UserDao {
    void insert();
}
```

- UserDao实现类

```
package org.cqipu.edu.dao;

import org.springframework.stereotype.Repository;

@Repository // 纳入bean管理
public class UserDaoForMySQL implements UserDao{
    @Override
    public void insert() {
        System.out.println("正在向mysql数据库插入User数据");
    }
}
```

- UserService

```
package org.cqipu.edu.service;
```

```
import org.cqipu.edu.dao.UserDao;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Service // 纳入bean管理
public class UserService {

    @Autowired // 在属性上注入
    private UserDao userDao;

    // 没有提供构造方法和setter方法。

    public void save(){
        userDao.insert();
    }
}
```

- 配置包扫描
 - spring-injection2.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">
    <context:component-scan base-package="org.cqipu.edu.dao,org.cqipu.edu.service"/>
</beans>
```

- 测试程序

```
@Test
public void testAutowired(){
    ApplicationContext applicationContext = new ClassPathXmlApplicationContext("spring-injection2.xml");
    UserService userService = applicationContext.getBean("userService", UserService.class);
    userService.save();
}
```

- 执行结果



以上构造方法和setter方法都没有提供，经过测试，仍然可以注入成功。

接下来，再来测试一下@Autowired注解出现在setter方法上：

- UserService

```
package org.cqipu.edu.service;

import org.cqipu.edu.dao.UserDao;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
```

```

@Service
public class UserService {

    private UserDao userDao;

    @Autowired
    public void setUserDao(UserDao userDao) {
        this.userDao = userDao;
    }

    public void save(){
        userDao.insert();
    }
}

```

执行结果

```

✓ 测试 已通过: 1共 1 个测试 - 264毫秒
D:\Java21\bin\java.exe ...
正在向mysql数据库插入User数据

```

我们再来看看能不能出现在构造方法上：

- UserService

```

package org.cqipu.edu.service;

import org.cqipu.edu.dao.UserDao;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
@Service
public class UserService {

    private UserDao userDao;

    @Autowired
    public UserService(UserDao userDao) {
        this.userDao = userDao;
    }

    public void save(){
        userDao.insert();
    }
}

```

执行结果

```

✓ 测试 已通过: 1共 1 个测试 - 250毫秒
D:\Java21\bin\java.exe ...
正在向mysql数据库插入User数据

```

再看看，这个注解能不能只标注在构造方法的形参上：

- UserService

```

package org.cqipu.edu.service;

import org.cqipu.edu.dao.UserDao;
import org.springframework.beans.factory.annotation.Autowired;

```

```
import org.springframework.stereotype.Service;
@Service
public class UserService {

    private UserDao userDao;

    public UserService(@Autowired UserDao userDao) {
        this.userDao = userDao;
    }

    public void save(){
        userDao.insert();
    }
}
```

执行结果

✓ 测试 已通过: 1共 1 个测试 - 285毫秒

D:\Java21\bin\java.exe ...
正在向mysql数据库插入User数据

还有更劲爆的，当有参数的构造方法只有一个时，@Autowired注解可以省略。

- UserService

```
package org.cqipu.edu.service;

import org.cqipu.edu.dao.UserDao;
import org.springframework.stereotype.Service;

@Service
public class UserService {

    private UserDao userDao;

    public UserService(UserDao userDao) {
        this.userDao = userDao;
    }

    public void save(){
        userDao.insert();
    }
}
```

执行结果

✓ 测试 已通过: 1共 1 个测试 - 270毫秒

D:\Java21\bin\java.exe ...
正在向mysql数据库插入User数据

当然，如果有多个构造方法，@Autowired肯定是不能省略的。

- UserService

```
package org.cqipu.edu.service;

import org.cqipu.edu.dao.UserDao;
import org.springframework.stereotype.Service;

@Service
```

```
public class UserService {

    private UserDao userDao;

    public UserService(UserDao userDao) {
        this.userDao = userDao;
    }

    public UserService(){

    }

    public void save(){
        userDao.insert();
    }
}
```

执行结果

D:\Java\jdk1\bin\java.exe ...

```
java.lang.NullPointerException: Cannot invoke "org.cqipu.edu.dao.UserDao.insert()" because "this.userDao" is null

    at org.cqipu.edu.service.UserService.save(UserService.java:21)
    at org.cqipu.edu.test.AnnotationTest.testAutowired(AnnotationTest.java:58) <25 个内部行>
```

进程已结束，退出代码为 -1

到此为止，我们已经清楚@Autowired注解可以出现在哪些位置了。

@Autowired注解默认是byType进行注入的，也就是说根据类型注入的，如果以上程序中，UserDao接口还有另外一个实现类，会出现问题吗？

- UserDaoForOracle，接口另一个实现类

```
package org.cqipu.edu.dao;
import org.springframework.stereotype.Repository;

@Repository // 纳入bean 管理
public class UserDaoForOracle implements UserDao{
    @Override
    public void insert() {
        System.out.println("正在向Oracle 数据库插入User 数据");
    }
}
```

当你写完这个新的实现类之后，此时 IDEA 工具已经提示错误信息了：

UserService.java D:\java_Spring6_study\spring6-ioc-annotation-demo09\src\main\java\org\cqipu\edu\service 1 个问题

无法自动装配。存在多个 'UserDao' 类型的 Bean。Beans:userDaoForMySQL (UserDaoForMySQL.java)userDaoForOracle (UserDaoForOracle.java) :13

错误信息中说：不能装配，UserDao这个Bean的数量大于1。

怎么解决这个问题呢？当然要byName，根据名称进行装配了。

@Autowired注解和@Qualifier注解联合起来才可以根据名称进行装配，在@Qualifier注解中指定Bean名称。

- UserDaoForOracle

```

package org.cqipu.edu.dao;
import org.springframework.stereotype.Repository;

@Repository // 这里没有给bean起名，默认名字是：userDaoForOracle
public class UserDaoForOracle implements UserDao{
    @Override
    public void insert() {
        System.out.println("正在向Oracle数据库插入User数据");
    }
}

```

- UserService

```

package org.cqipu.edu.service;

import org.cqipu.edu.dao.UserDao;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.stereotype.Service;

@Service
public class UserService {

    private UserDao userDao;

    @Autowired
    @Qualifier("userDaoForOracle") // 这个是bean的名字。
    public void setUserDao(UserDao userDao) {
        this.userDao = userDao;
    }

    public void save(){
        userDao.insert();
    }
}

```

执行结果

✓ 测试 已通过: 1共 1 个测试 - 262毫秒

D:\Java21\bin\java.exe ...

正在向Oracle数据库插入User数据

总结:

- @Autowired 注解可以出现在：属性上、构造方法上、构造方法的参数上、setter 方法上。
- 当带参数的构造方法只有一个，@Autowired 注解可以省略。
- @Autowired 注解默认根据类型注入。如果要根据名称注入的话，需要配合 @Qualifier 注解一起使用。

@Resource

@Resource 注解也可以完成非简单类型注入。那它和 @Autowired 注解有什么区别？

- @Resource 注解是 JDK 扩展包中的，也就是说属于 JDK 的一部分。所以该注解是标准注解，更加具有通用性。(JSR-250 标准中制定的注解类型。JSR 是 Java 规范提案。)
- @Autowired 注解是 Spring 框架自己的。
- @Resource 注解默认根据名称装配 byName，未指定 name 时，使用属性名作为 name。通过 name 找不到的话会自动启动通过类型 byType 装配。

- @Autowired 注解默认根据类型装配 byType，如果想根据名称装配，需要配合 @Qualifier 注解一起用。
- @Resource 注解用在属性上、setter 方法上。
- @Autowired 注解用在属性上、setter 方法上、构造方法上、构造方法参数上。

@Resource 注解属于 JDK 扩展包，所以不在 JDK 当中，需要额外引入以下依赖：【如果是 JDK8 的话不需要额外引入依赖。高于 JDK11 或低于 JDK8 需要引入以下依赖。】

- 如果是Spring6+版本请使用这个依赖

```
<dependency>
  <groupId>jakarta.annotation</groupId>
  <artifactId>jakarta.annotation-api</artifactId>
  <version>2.1.1</version>
</dependency>
```

一定要注意：如果你用Spring6，要知道Spring6不再支持JavaEE，它支持的是JakartaEE9。（Oracle把JavaEE贡献给Apache了，Apache把JavaEE的名字改成JakartaEE了，大家之前所接触的所有的 javax.* 包名统一修改为 jakarta.*包名了。）

- 如果你是spring5-版本请使用这个依赖

```
<dependency>
  <groupId>javax.annotation</groupId>
  <artifactId>javax.annotation-api</artifactId>
  <version>1.3.2</version>
</dependency>
```

@Resource注解的源码如下：

```
4  @Target({ElementType.TYPE, ElementType.FIELD, ElementType.METHOD})
5  @Retention(RetentionPolicy.RUNTIME)
6  @Repeatable(Resources.class)
7  public @interface Resource {
8      String name() default ""; name属性用来接收bean的名称
9
10     String lookup() default "";
11
12     Class<?> type() default Object.class;
13
14     AuthenticationType authenticationType() default Resource.AuthenticationType.CONTAINER;
15
16     boolean shareable() default true;
```

老杜

测试一下：

- 给这个UserDaoForOracle起名xyz



```
package org.cqipu.edu.dao;
import org.springframework.stereotype.Repository;
@Repository("xyz")
public class UserDaoForOracle implements UserDao{
    @Override
    public void insert() {
        System.out.println("正在向Oracle数据库插入User数据");
    }
}
```

- 在UserService中使用Resource注解根据name注入



```
package org.cqipu.edu.service;

import org.cqipu.edu.dao.UserDao;
import org.springframework.stereotype.Service;
import jakarta.annotation.Resource;

@Service
public class UserService {

    @Resource(name = "xyz")
    private UserDao userDao;

    public void save(){
        userDao.insert();
    }
}
```

执行结果

```
D:\Java21\bin\java.exe ...
正在向Oracle数据库插入User数据
```

我们把UserDaoForOracle的名字xyz修改为用户dao，让这个Bean的名字和UserService类中的UserDao属性名一致：

- UserDaoForOracle



```
package org.cqipu.edu.dao;
import org.springframework.stereotype.Repository;

@Repository("userDao")
public class UserDaoForOracle implements UserDao{
    @Override
    public void insert() {
        System.out.println("正在向Oracle数据库插入User数据");
    }
}
```

- UserService类中Resource注解并没有指定name



```
package org.cqipu.edu.service;

import org.cqipu.edu.dao.UserDao;
import org.springframework.stereotype.Service;
import jakarta.annotation.Resource;

@Service
```

```
public class UserService {

    @Resource
    private UserDao userDao;

    public void save(){
        userDao.insert();
    }
}
```

执行结果

✓ 测试 已通过: 1共 1 个测试 - 271毫秒

D:\Java21\bin\java.exe ...

正在向Oracle数据库插入User数据

通过测试得知，当@Resource注解使用时没有指定name的时候，还是根据name进行查找，这个name是属性名。

接下来把UserService类中的属性名修改一下：

- UserService的属性名修改为userDao2

```
package org.cqipu.edu.service;

import org.cqipu.edu.dao.UserDao;
import org.springframework.stereotype.Service;
import jakarta.annotation.Resource;

@Service
public class UserService {

    @Resource
    private UserDao userDao2;

    public void save(){
        userDao2.insert();
    }
}
```

执行结果

D:\Java21\bin\java.exe ...

4月 04, 2026 8:53:10 下午 org.springframework.context.support.AbstractApplicationContext refresh

警告: Exception encountered during context initialization - cancelling refresh attempt: org.springframework.beans.factory.BeanCreationException: Error creating bean with name 'userService': Injection of resource dependencies failed

org.springframework.beans.factory.BeanCreationException: Error creating bean with name 'userService': Injection of resource dependencies failed

根据异常信息得知：显然当通过name找不到的时候，自然会启动byType进行注入。以上的错误是因为UserDao接口下有两个实现类导致的。所以根据类型注入就会报错。

我们再来看@Resource注解使用在setter方法上可以吗？

- UserService添加setter方法并使用注解标注

```
package org.cqipu.edu.service;

import org.cqipu.edu.dao.UserDao;
import org.springframework.stereotype.Service;
import jakarta.annotation.Resource;

@Service
```

```

public class UserService {

    private UserDao userDao;

    @Resource
    public void setUserDao(UserDao userDao) {
        this.userDao = userDao;
    }

    public void save(){
        userDao.insert();
    }
}

```

注意这个 setter 方法的方法名，setUserDao 去掉 set 之后，将首字母变小写 userDao，userDao 就是 name 执行结果：

✓ 测试 已通过: 1共 1 个测试 - 365毫秒

D:\Java21\bin\java.exe ...

正在向Oracle数据库插入User数据

当然，也可以指定 name:

- UserService

```

package org.cqipu.edu.service;

import org.cqipu.edu.dao.UserDao;
import org.springframework.stereotype.Service;
import jakarta.annotation.Resource;

@Service
public class UserService {

    private UserDao userDao;

    @Resource(name = "userDaoForMySQL")
    public void setUserDao(UserDao userDao) {
        this.userDao = userDao;
    }

    public void save(){
        userDao.insert();
    }
}

```

执行结果：

✓ 测试 已通过: 1共 1 个测试 - 285毫秒

D:\Java21\bin\java.exe ...

正在向mysql数据库插入User数据

一句话总结 @Resource 注解：默认 byName 注入，没有指定 name 时把属性名当做 name，根据 name 找不到时，才会 byType 注入。byType 注入时，某种类型的 Bean 只能有一个。

全注解式开发

所谓的全注解开发就是不再使用spring配置文件了。写一个配置类来代替配置文件。

- 配置类代替spring配置文件

- o Spring6Configuration.java

```
package org.cqipu.edu.config;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.ComponentScans;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan({"org.cqipu.edu.dao", "org.cqipu.edu.service"})
public class Spring6Configuration {
}
```

编写测试程序：不再new ClassPathXmlApplicationContext()对象了。

```
package org.cqipu.edu.test;
import org.cqipu.edu.config.Spring6Configuration;
import org.cqipu.edu.service.UserService;
import org.junit.Test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class AnnotationTest {
    @Test
    public void testNoXml(){
        ApplicationContext applicationContext = new AnnotationConfigApplicationContext(Spring6Configuration.class);
        UserService userService = applicationContext.getBean("userService", UserService.class);
        userService.save();
    }
}
```

✓ 测试 已通过: 1共 1 个测试 - 273毫秒

D:\Java21\bin\java.exe ...

正在向mysql数据库插入User数据